GRADUATE RESEARCH ASSISTANT (GRA) & GRADUATE ASSISTANT (GA) STUDENT LIFECYCLE MANAGEMENT SYSTEM FOR URESEARCH 2.0

by

ABDUL HAZIQ BIN ABDUL FAROUK

22007428

Interim report submitted in partial fulfilment of

The requirements for the

Degree of

Bachelor of Computer Science/Information Systems/Information Technology (Hons)

FYP I Semester 1 and Final Year

Universiti Teknologi PETRONAS

32610 Seri Iskandar

Perak Darul Ridzuan

CERTIFICATION OF APPROVAL

GRADUATE RESEARCH ASSISTANT (GRA) & GRADUATE ASSISTANT (GA) STUDENT LIFECYCLE MANAGEMENT SYSTEM FOR URESEARCH 2.0

by

ABDUL HAZIQ BIN ABDUL FAROUK
22007428

Interim Report submitted to the Programme
Name
Universiti Teknologi PETRONAS
in partial fulfilment of the requirement for the
Bachelor of Computer Science/Information Systems/Information Technology (Hons)

Approved by,

____
Ts Dr. Savita K. Sugathan

UNIVERSITI TEKNOLOGI PETRONAS
BANDAR SERI ISKANDAR, PERAK
FYP I Semester 1 and Final Year

CERTIFICATION OF ORIGINALITY

This is to certify that I am responsible for the work submitted in this project, that the original work is my own except as specified in the references and acknowledgements, and that the original work contained herein have not been undertaken or done by unspecified sources or persons.

ABDUL HAZIQ BIN ABDUL FAROUK

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor for the guidance, patience, and constructive feedback given throughout the proposal defence and interim reporting stages of this project. I would also like to thank the Centre for Graduate Studies and the UResearch 2.0 project team for their willingness to share the operational context behind the current GRA/GA process, which shaped much of the requirement analysis presented in this report. Finally, I am grateful to my family and friends for their continued encouragement over the course of this project.

ABSTRACT

The Centre for Graduate Studies (CGS) at Universiti Teknologi PETRONAS currently manages the entire Graduate Research Assistant (GRA) and Graduate Assistant (GA) student lifecycle through paper forms, email correspondence, and personal spreadsheets. This manual arrangement leaves students and staff with no real-time visibility into where an application stands, no automated tracking of stage gate deadlines and grace periods, and no consolidated view of monthly allowance eligibility. This project, developed as an enhancement module for UResearch 2.0, proposes a centralized, web-based GRA/GA Student Lifecycle Management System intended to replace this manual workflow with a single, AI-assisted platform. The system is organised around three core modules which are first Module 1 that handles GRA application, multi-level approval, and stage gate monitoring with automated suspend, terminate, and recover logic. Next is Module 2 that handles GA application, eligibility screening, research centre interviews, and committee approval and lastly Module 3 that consolidates allowance eligibility into a single live report, replacing what was previously a missing, manually compiled record. The system is supported by role-based visual dashboards for six stakeholder roles, and an artificial-intelligence layer comprising a conversational FAQ chatbot that answers student status queries using live application data. The system is built using PHP 8 and MySQL on a XAMPP environment, with a Bootstrap 5 and Chart.js frontend, PHPMailer for automated notifications, Dompdf for letter generation, and an LLM API for the chatbot. Development follows an Interactive Waterfall model, which keeps the traditional linear phases of requirement analysis, system design, implementation, testing, and deployment, while allowing the team to return directly to any earlier phase the moment an issue is discovered, without needing to repeat multiple sprint cycles. The resulting prototype is expected to demonstrate measurable gains in process transparency, a reduction in manual approval steps, accurate stage gate and allowance logic, and a chatbot capable of answering common student status questions correctly.

TABLE OF CONTENT

LIST OF TABLES

LIST OF FIGURES

LIST OF APPENDICES

LIST OF ABBREVIATIONS

Abbreviation	Meaning
CGS	Centre for Graduate Studies
GRA	Graduate Research Assistant
GA	Graduate Assistant
GRS Exec	Graduate Research Studies Executive
RPD	Research Proposal Defence
BPR	Business Process Reengineering
LLM	Large Language Model
AI	Artificial Intelligence
UAT	User Acceptance Testing
SUS	System Usability Scale
ERD	Entity-Relationship Diagram
FYP	Final Year Project
UTP	Universiti Teknologi PETRONAS

CHAPTER 1: INTRODUCTION

1.1 Background of Study

Every semester, a large number of postgraduate students work as Graduate Research Assistants (GRA) and Graduate Assistants (GA) at Universiti Teknologi PETRONAS (UTP). These roles are crucial to the university's research ecosystem since GA students assist with teaching and academic operations, while GRA students assist supervisors on sponsored research projects. However, the Center for Graduate Studies (CGS) still oversees a completely manual procedure for managing this population. Stage gate deadlines are monitored using individual staff members' personal spreadsheets, applications are submitted using paper or online but unstructured forms, and approvals are communicated via email between the Supervisor, Admin, GRS Exec, and Senior Director.

When there were fewer GRA and GA students, this system was sufficient. However, as UResearch 2.0 has increased its research funding activities, the volume of applications, approvals, and continuing stage gate milestones has increased to the point that the manual procedure is no longer feasible. Currently, there is not a centralized platform that allows staff members or students to instantly monitor the status of an application, the approaching stage gate deadline, or the status of a student's allowance. By suggesting a specific GRA/GA Student Lifecycle Management System for UResearch 2.0, our proposal directly addresses that need.

1.1.1 The Manual Application and Approval Gap

- GRA/GA applications and multi-level approvals (Admin → GRS Exec → Supervisor → Senior Director) currently rely on physical forms and email correspondence.
- Neither students nor staff have real-time visibility into where an application currently stands in the approval chain.

1.1.2 Limitations of Current Tracking Methods

- Stage gate deadlines and grace periods are manually tracked using spreadsheets, with no automatic reminders for students or staff.
- There is no reliable way to determine which stage or accountable actor is causing delays in the entire process.

1.1.3 Fragmented Data and the Absence of Centralization

- Student data, supporting documentation, and approval records are currently stored in emails, physical files, and personal spreadsheets.
- The lack of a single source of truth makes it difficult to report, forecast, or audit accurately.
.

1.1.4 Why This Project Is Needed Now

The timeframe of this project is directly related to the expansion of UResearch 2.0 as a platform. As more research grants are awarded and more supervisors enlist GRA and GA assistance, the

CGS personnel in charge of organizing applications and stage gates has not expanded at the same rate. What was once a manageable manual workload for a small number of staff members has become a bottleneck, especially near the end of each semester when numerous students reach a stage gate deadline at the same time. Left untreated, this bottleneck is only projected to grow worse as UResearch 2.0 grows, making now an acceptable time to implement a centralized system rather than waiting until the manual method fails completely.

1.2 Problem Statement

Building on the background discussed above, four specific problems can be identified that together justify the need for this project.

1.2.1 Manual, Paper-Based Process

Application and approval handling is time-consuming and error-prone because it relies on physical forms and email correspondence rather than a structured digital workflow. Every stage of the process from initial submission to final Senior Director sign-off depends on staff manually forwarding information rather than the system routing it automatically.

1.2.2 Lack of Process Visibility

Students and staff currently have no real-time means to follow the status of an application or approval, let alone identify who or what is causing the delay. A student who has submitted an application has no way of knowing if their file is being reviewed by the Supervisor, verified by the Administrator, or approved by the Senior Director unless they contact CGS directly.

1.2.3 Fragmented and Disconnected Data

Student data, supporting documentation, and approval records are dispersed across traditional and manual sources, with no centralization. This fragmentation makes it difficult to create unified reports and increases the likelihood that a document or record is lost between stages.

1.2.4 No Early Warning Before Deadlines Are Missed

Manual deadline tracking provides no indication of which students are likely to miss the next stage gate until it has occurred. By the time a missed deadline is discovered, the grace period may have already expired, leaving little time for corrective action before a student's allowance is suspended or their candidacy is terminated.

1.3 Objectives

In order to directly address the problems identified above, this project sets out to achieve the following three objectives:

- To design and develop a centralized online system for GRA/GA application, document submission, and multi-level approval.

- To implement an automated Stage Gate monitoring system for tracking research milestones, deadlines, and allowance status.
- To provide role-based dashboards and AI-powered insights for real-time visibility across all stakeholders.

1.4 Scope of Study

The scope of this project is organised around three functional areas which are system automation, data management and visualization, and artificial intelligence. Each area is described in turn below.

1.4.1 System Automation Scope

- Covers the full GRA/GA/Extension application lifecycle, from initial submission through to final approval.
- Automated multi-level approval routing, following the chain Admin → GRS Exec → Supervisor → Senior Director
- Automated document tracking and reminder notifications for any missing or incorrect submissions.
- Automated stage gate deadline tracking, grace period reminders, and allowance status updates covering suspend, recover, and terminate logic.
- Automated generation of Offer and Admission Letters immediately upon approval.

1.4.2 Data Management and Visualization Scope

- A structured digital form covering Sections B to F (Project Details, GRA Details, Academic Qualification, Working Experience, Publications) with data validation applied on input.
- Design of a relational database structure to store application data, documents, approval status, stage gates, and allowance records.
- Document metadata storage, including file type, upload date, version, and submission status, linked to each application record.
- Role-based dashboards limited to the five core system roles: Student, Admin, GRS Exec, Supervisor, and Senior Director, in addition to a Research Centre view.
- Visual analytics covering the application status funnel, document completeness, stage gate progress, delay-by-actor analysis, and allowance eligibility.

1.4.3 Artificial Intelligence Scope

- A conversational chatbot for student status queries, built on top of a large language model (LLM) API and fed with live application data rather than a static knowledge base.

Taken together, these three domains create a bounded but complete system: it is not intended to replace UResearch 2.0's broader grant-management responsibilities, but rather to complement them as the dedicated module responsible for the GRA/GA student lifetime.

1.5 Significance of Study

The significance of this project can be viewed from three perspectives which are the student, the administrative personnel, and the institution in general. For the student, the system substitutes an unclear, email-driven procedure with a single dashboard that displays exactly where their application is, what documents are still outstanding, and whether their allowance is guaranteed for the next month. The system relieves administrative staff, including Admin, GRS Exec, Supervisors, and the Senior Director, of the burden of manually tracking dozens of students across personal spreadsheets, instead surfacing overdue cases and bottlenecks automatically via the dashboards described in Chapter 4.

At the institutional level, the project is significant because it provides, for the first time, a single reliable data source describing the entire GRA/GA population whcih are how many students are active, how many are at risk of missing a stage gate, and how much allowance is paid out each month. This kind of institutional visibility was previously impossible to achieve without a time-consuming manual audit of many spreadsheets and email threads, and it is expected to help with resource planning in UResearch 2.0 in the future.

1.6 Organisation of the Report

The remainder of this report is divided into four further chapters. Chapter 2 examines the existing literature on business process reengineering in higher education administration and uses it to identify the specific gap that this project fills. Chapter 3 details the project's methodology, including the development model, tools and technologies, system architecture, database design, process flow for all three modules, artificial intelligence integration, and project timetable. Chapter 4 describes the system's user-facing design, which includes role-based dashboards, the data needed to power them, a testing plan, and risk management considerations. Chapter 5 concludes the study and discusses future work beyond the scope of the project.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview

Before deciding on a system design, it is necessary to consider how similar administrative tasks have been digitalized elsewhere, particularly in higher education. There is no published study that directly examines a GRA/GA lifecycle system, although other studies discuss how business process reengineering (BPR) is utilized for similar university administrative duties. This chapter examines these research and applies them, along with some basic BPR theory and a few supporting studies on AI chatbots, to illustrate the gap that this project aims to address.

2.2 Business Process Reengineering in Higher Education Administration

2.2.1 Foundations of Business Process Reengineering

Before evaluating higher-education-specific studies, it is important to define BPR as a methodology. Davenport and Short (1990) define BPR as the fundamental study and redesign of workflows and processes, proposing that information technology should be viewed as a tool for reshaping how labor is organized rather than just automating current paper-based activities. This principle underpins all three of the higher-education case studies examined below, and it is why this project prioritizes automation of the GRA/GA procedure over a digitized replica of the existing paper form as its primary design goal.

2.2.2 Critical Success Factors for BPR in Higher Education Institutions

Ahmad, Francis, and Zairi (2007) looked at what made BPR succeed across three private higher education institutions in Malaysia. They found seven factors that mattered which are teamwork and quality culture, a proper quality management system with fair rewards, effective change management, a less bureaucratic and more participative structure, IT/IS readiness, effective project management, and having enough financial resources. Out of these seven, two apply quite directly to this project. IT/IS readiness matches how the GRA/GA system is being built entirely on tools the team already knows, namely PHP, MySQL, and XAMPP, which keeps the technical risk low enough to realistically finish within a two-semester FYP. Effective project management matches the use of the Interactive Waterfall model in Section 3.2, where the project is broken down into clear phases with a way to return to an earlier phase if something is found to be wrong, instead of having to redo the whole approval process from scratch. The other five factors are less relevant here, since they mostly apply to a real institution rolling out BPR across departments with actual staff, budget, and reward structures, which is not really the situation for a single-developer FYP. Even so, this shows that the two factors this project does lean on, having the right IT/IS fit and a properly managed development process, are the same two that Ahmad et al. (2007) found to matter in a Malaysian higher education setting, which gives some support to the overall approach taken here.

2.2.3 Simulation-Driven Workflow Automation

Renna and Colonnese (2025) conducted a Business Process Reengineering (BPR) study at the University of Basilicata to improve the workflow for teaching assignment management. One aspect that makes their study particularly valuable is the structured approach they adopted. They first analysed the existing **AS-IS** process by modelling it with BPMN 2.0 to identify bottlenecks and inefficiencies. Based on this analysis, they designed a new **TO-BE** process and evaluated it using discrete event simulation before implementation. Rather than implementing the redesigned workflow immediately, they tested it first to determine whether it could achieve the intended improvements.

The simulation results showed that automating the routing and approval activities significantly reduced manual processing time, improved resource allocation, and minimised delays throughout the workflow. These findings are relevant to the proposed GRA/GA Management System because it also involves multiple levels of approval. Similar to the study, this project replaces the existing email-based approval process with a structured digital workflow involving the Administrator, GRS Executive, Supervisor, and Senior Director. Although this project does not include discrete event simulation due to the limited scope and duration of a two-semester Final Year Project, it adopts the same underlying principle that automating workflow routing can reduce unnecessary delays, minimise manual intervention, and improve the overall efficiency of the approval process.

2.2.4 BPR for Official Trip Procedures

Pasaribu, Anggadwita, Hendayani, Kotjoprayudi, and Apiani (2021) examined the Business Process Reengineering (BPR) of the official trip approval process at Telkom University, Indonesia. Before the redesign, the process consisted of a combination of manual and online procedures that were highly centralised, causing applications to pass through multiple approval levels before any decision could be made. To improve the workflow, the researchers adopted a mixed-method approach by analysing processing performance before and after the redesign, while also conducting interviews and focus group discussions with university staff to better understand the existing challenges.

Based on their findings, they redesigned the workflow into a fully online and decentralised process supported by a transparent monitoring system that allowed users to track the status of each application throughout the approval chain. The redesigned process resulted in faster processing times, reduced paper usage, and more efficient utilisation of staff resources.

This study is particularly relevant to the proposed GRA/GA Management System because both processes involve a multi-level approval workflow that traditionally relies on paper documents and email communication. Similar to the official trip approval process, GRA/GA applications require approvals from several stakeholders before they can be finalised. Together with the study by Renna and Colonnese (2025), this research demonstrates how BPR can be applied to improve recurring administrative workflows by automating routing, approvals, and application tracking. These findings support the design of the proposed system, which aims to replace the current manual approval process with a structured digital workflow that improves efficiency, transparency, and overall process management.

2.2.5 BPR for a One-Time Application Process

Saad, Zaini, and Alwahaibi (2024) examined a different type of administrative process involving Medical Assistance Fund (MAF) applications managed by the Medical Social Work Unit in a public hospital. Unlike the previous studies, the delays in this process were mainly caused by the increasing number of applications and the continued reliance on manual procedures. To address these issues, the researchers adopted a hybrid approach known as B2Lean, which combines Business Process Reengineering (BPR) with Lean Thinking, and proposed the Patient Assistance Management System to support the redesigned workflow.

However, the nature of this process differs significantly from the GRA/GA management process. The MAF application is essentially a one-time administrative process. Once an application is approved and the financial assistance is disbursed, the system's role is largely complete, with no further monitoring of the applicant required. In contrast, GRA/GA students must be continuously monitored throughout their candidature. Their progress is evaluated against multiple stage gates, such as the Research Proposal Defence (RPD), publication requirements, and other academic milestones. Failure to meet these requirements may result in consequences such as allowance suspension or even termination of their assistantship.

This distinction highlights an important research gap. While the system proposed by Saad et al. (2024) successfully improves the efficiency of the initial application process, it does not address the need for continuous post-approval monitoring. Therefore, it would not be appropriate to classify this study together with the works of Renna and Colonnese (2025) or Pasaribu et al. (2021), which primarily focus on automating workflow routing and approval processes. Instead, Saad et al. (2024) represent a separate category of BPR implementation, one that improves application processing but does not support long-term monitoring after approval. This limitation further justifies the need for the proposed GRA/GA Monitoring System, which extends beyond application management by supporting continuous monitoring throughout a student's research journey.

2.3 Research Gap and Research Direction

No published study directly addresses a GRA/GA-specific lifecycle system. The closest similar work is general business process reengineering applied to higher education administration. The table below outlines the specific limitations noted in each stream of earlier work and explains how this project's proposed approach addresses those limitations

Stream of Existing Work	Key Source(s)	Limitation Identified	This System's Response
Routing/approval automation BPR studies	Renna & Colonnese (2025); Pasaribu et al. (2021)	Show that automating multi-level routing saves manual handling time, but do not cover postgraduate research-funding lifecycles with milestone-based allowance suspension, recovery, or termination..	A stage gate engine with deadline, grace period, allowance suspension, candidacy termination, and recovery logic built for GRA/GA milestones.
One-time application-intake BPR studies	Saad, Zaini & Alwahaibi (2024)	Digitises intake and approval, but the process stops once approved with no ongoing monitoring of the applicant afterwards.	Goes past one-time approval into continuous stage gate and allowance monitoring, visible to six stakeholder roles.
Commercial scholarship/grant platforms	Industry practice	Centralise intake but are generic, not built for a university's own multi-level approval chain, and give no milestone or predictive data.	Built specifically around UTP's own forms and interview flow, plus an AI chatbot and at-risk flagging.

Taken together, this comparison shows that the proposed system is not simply digitizing an existing form but it is extending BPR principles into a domain, postgraduate research-funding lifecycle management, that has not previously been addressed in the reviewed literature.

2.4 Comparative Summary of Existing Approaches

To further clarify the positioning of this project, Table 1 below places the reviewed literature and typical commercial scholarship or grant-management platforms alongside the proposed system, across a set of features that this project considers essential to GRA/GA lifecycle management.

Feature	Routing Automation BPR Studies	Application-Intake BPR Studies	Commercial Grant Platforms	Proposed System
Digitized application & approval routing	Yes	Yes	Yes	Yes

Multi-level, institution-specific approval chain	Yes	Partial	No	Yes
Milestone-based stage gate tracking with grace periods	No	No	No	Yes
Automated allowance suspend/terminate/recover logic	No	No	No	Yes
Ongoing post-approval, multi-role monitoring	No	No	Partial	Yes
Consolidated live allowance eligibility reporting	No	No	Partial	Yes
Role-based analytical dashboards (6 roles)	No	No	Partial	Yes
AI-powered conversational status assistant	No	No	No	Yes

This comparison reinforces the conclusion reached in Section 2.3 while existing approaches address individual aspects of the problem, none combine institution-specific multi-level approval, milestone-based stage gate logic, and AI-assisted status reporting into a single, purpose-built system for postgraduate research funding lifecycle management.

2.5 Supporting Literature: AI-Assisted Student Services

Apart from Business Process Reengineering (BPR), this project also draws on recent studies that explore the use of large language models (LLMs) to support student enquiries. Alabbas, Alomar, and Tayseer (2024) developed an AI chatbot for technical and vocational education institutions, demonstrating that it could effectively answer routine student enquiries while reducing the workload of administrative staff. Similarly, Sajja, Sermet, Cikmaz, Cwiertny, and Demir (2024) proposed an AI assistant capable of retrieving real-time learner information instead of relying solely on a fixed set of frequently asked questions (FAQs). These studies support the approach adopted in Section 3.7 of this project, where the GRA/GA chatbot is designed to retrieve live information from the system database, including application status, submitted documents, and allowance eligibility, rather than providing only static, pre-written responses. This allows students to receive more accurate, personalised, and up-to-date information whenever they interact with the chatbot.

2.6 Supporting Literature: Development Methodology

The selection of the Interactive Waterfall model described in Section 3.2 is also supported by the work of Balaji and Murugaiyan (2012), who compared the Waterfall, V-Model, and Agile software development approaches. Their study concluded that the traditional Waterfall model is well suited to projects with clearly defined and stable requirements. However, they also highlighted one of its main limitations: once a phase has been completed, making changes later in the development process is difficult because there is no straightforward way to return to an earlier stage. This limitation influenced the choice of the Interactive Waterfall model for this project. Unlike the traditional Waterfall approach, the Interactive Waterfall model allows the development team to revisit previous phases whenever necessary, making it easier to refine requirements, improve the system design, and address issues identified during later stages of development, as discussed in Section 3.2.

2.7 Chapter Summary

This chapter reviewed the literature related to Business Process Reengineering (BPR) in higher education administration, covering routing and approval automation, application intake processes, commercial grant management platforms, fundamental BPR concepts, AI chatbot technologies, and software development methodologies. The review shows that previous studies have successfully addressed workflow automation and one-time application processing as separate areas. However, none of the existing solutions provide a complete system that supports the continuous management of GRA/GA students throughout their candidature. Key requirements such as milestone-based stage gates, allowance suspension and recovery, termination management, and ongoing monitoring by multiple administrative roles remain largely unaddressed. This identified gap forms the basis for the system design presented in the next chapter, which aims to integrate these features into a single lifecycle management system.

CHAPTER 3: METHODOLOGY

3.1 Introduction

This chapter describes how the GRA/GA Student Lifecycle Management System was planned and is being developed. It begins with the chosen development methodology, followed by the tools and technologies selected for implementation, the system's layered architecture, the design of its underlying database, the process flow of each of its three modules, its artificial intelligence integration, and finally the project timeline covering both FYP 1 and FYP 2.

3.2 Development Methodology: Interactive Waterfall Model

This project used the Interactive Waterfall model as its method. Unlike traditional Waterfall approaches, which proceed strictly in one direction, and unlike Agile or Scrum, which rely on many repeating sprint cycles, the Interactive Waterfall model maintains the familiar linear phases of Requirement Analysis, System Design, Implementation, Testing, and Deployment while allowing the team to return to any earlier phase the moment an issue is discovered, without having to cycle through multiple sprints.

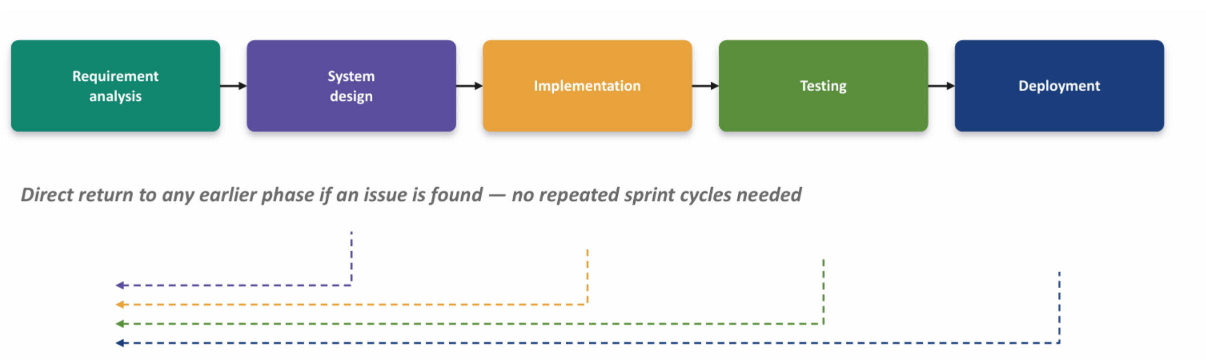


Figure 1: Interactive Waterfall Development Model

This model was selected because changes and improvements may arise during the development and testing stages. For example, if defects are identified during testing, the development team can immediately return to the implementation or system design phase to resolve the issue before continuing. Similarly, if stakeholders request changes to the system requirements or are not satisfied with certain functionalities during testing, the team can revisit the requirement analysis or design phase without restarting the entire development process. This flexibility makes the Interactive Waterfall model more suitable than the traditional Waterfall model, while avoiding the overhead of managing multiple Agile or Scrum sprint cycles.

Table 2 below compares the three approaches directly, to make the reasoning behind this choice explicit.

Aspect	Traditional Waterfall	Agile / Scrum	Interactive Waterfall (Chosen)
Phase structure	Strictly linear, one direction only	Repeating sprint cycles	Linear phases, with a direct return path to any earlier phase
Handling a discovered issue	Requires restarting the whole project or a formal change request	Requires waiting for or repeating a sprint cycle	Team returns directly to the relevant earlier phase without a full sprint
Suitability for a solo FYP student	Rigid; risky if requirements shift mid-project	Overhead of sprint ceremonies may not suit a single developer	Keeps a clear plan while remaining adaptable to a single developer's pace
Best fit for this project	Not ideal, given the intricate stage gate logic	Possible, but heavier than necessary for a two-semester solo project	Selected approach

3.3 Tools and Technologies

The tools selected for this project were chosen for their maturity, low licensing cost, and suitability for a locally hosted academic prototype. Table 2 summarizes the tools used across each layer of the system.

Category	Tool(s)	Purpose
Development & Environment	PHP 8, XAMPP, Git + GitHub	Backend development, local web server environment, and version control
Data Management	MySQL, phpMyAdmin	Store and manage application data
Backend Services	PHP Sessions + password hashing, PHPMailer (SMTP), Dompdf	User authentication, session management, email notifications, and PDF letter generation
Frontend & Design	HTML5, CSS3, Bootstrap 5, JavaScript, Chart.js, Figma, Draw.io/StarUML	Responsive user interface, data visualization, UI prototyping, and system modelling
Artificial Intelligence	LLM API (OpenAI / Gemini)	Powers the conversational status chatbot by answering

Category	Tool(s)	Purpose
		queries related to allowance applications and approval status

3.3.1 Tool Selection Rationale

PHP and MySQL were chosen because they are free to use, widely documented, and already familiar to the development team through undergraduate coursework. This made it easier to develop the system within the project timeline. Bootstrap 5 and Chart.js were selected for the frontend because they provide ready-made responsive components and charts, allowing the dashboard to be developed more quickly with less custom coding. PHPMailer and Dompdf were chosen because they work well with a PHP-based system and can be integrated without using additional frameworks, keeping the system simple and lightweight. Finally, an LLM API was selected for the chatbot instead of developing a custom AI model because it provides accurate conversational responses without requiring the team to collect and train a large dataset. This significantly reduced the complexity of the project and made it more practical to complete within the two-semester FYP period.

3.4 System Architecture

The system is developed as one of six member modules within the wider UResearch 2.0 platform, which is a web-based portal serving the Centre for Graduate Studies' administrative needs. The

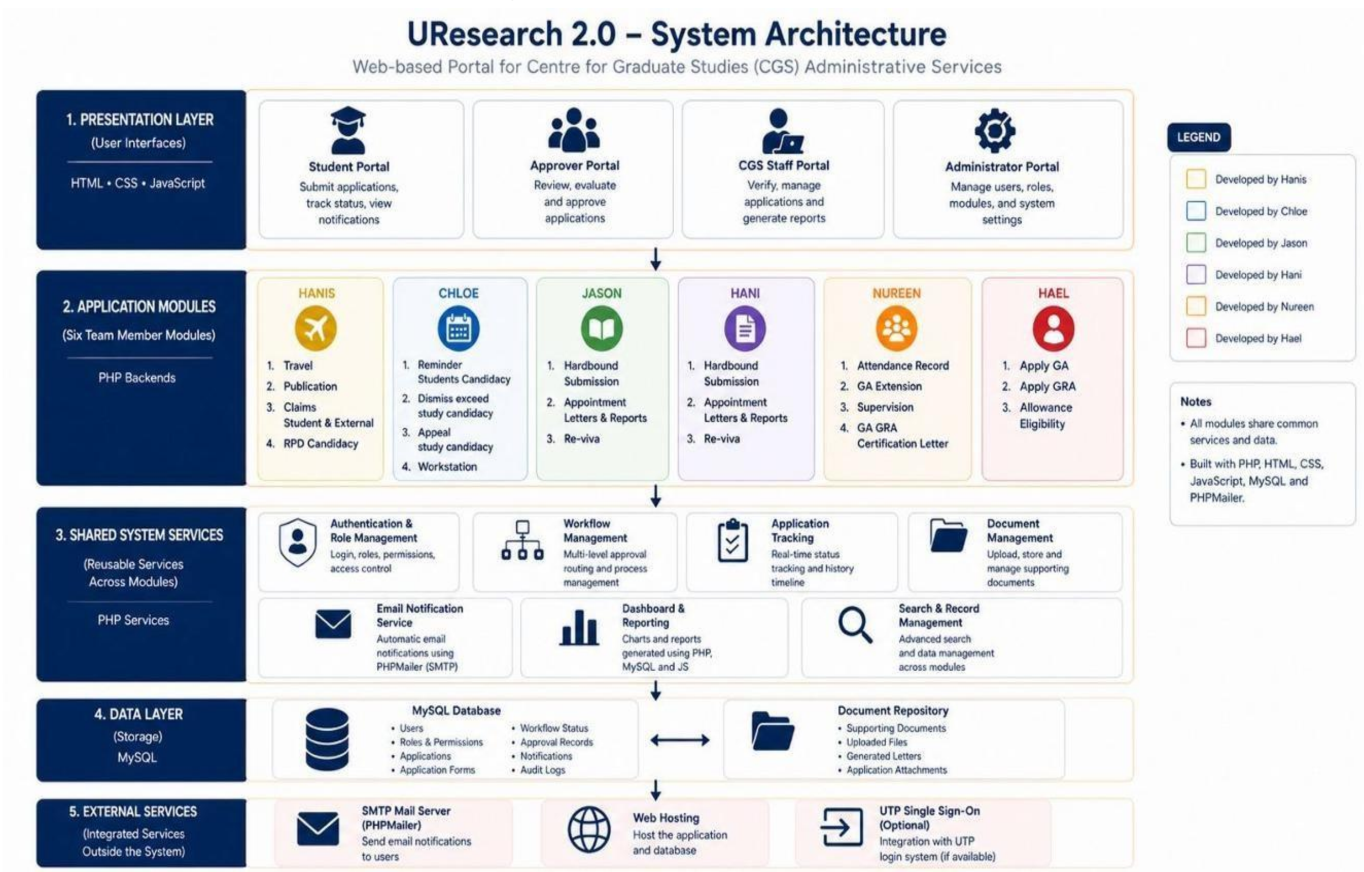


Figure 2: UResearch 2.0 System Architecture (Six-Module Overview)

overall platform architecture is organised into five layers: the presentation layer, the application modules layer, the shared system services layer, the data layer, and the external services layer.

Within the overall platform, this project focuses on the GRA/GA module. The module is responsible for the **Apply GA and Apply GRA**, **GRA Extension**, **Supervision**, **GA/GRA Candidacy Approval Letter**, and **Allowance Eligibility** functions shown in the system architecture. It also uses the platform's shared services, including **Authentication and Role Management**, **Workflow Management**, **Application Tracking**, **Document Management**, **Email Notification**, **Dashboard and Reporting**, and **Search and Record Management**. All of these modules and services share the same MySQL database and document repository, allowing data to be managed consistently across the platform.

Looking specifically at the GRA/GA module itself, its internal architecture can be described in four layers, sitting beneath the shared platform services described above.

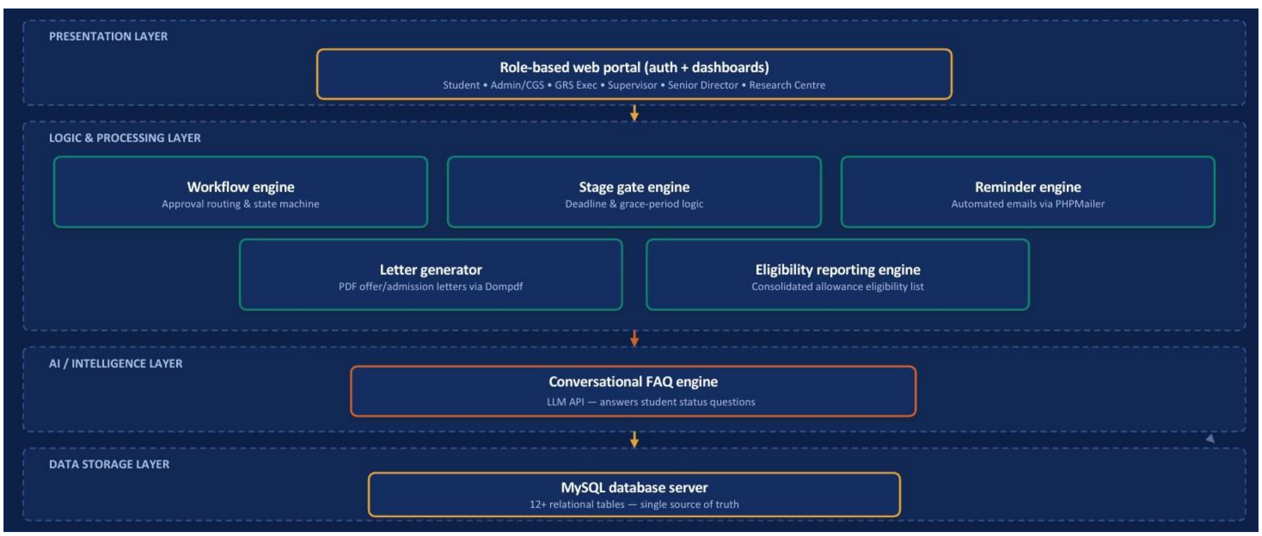


Figure 3: Layered System Architecture Diagram

Component	Responsibility
Workflow engine	Routes GRA/GA applications through the approval chain and updates status at every step
Stage gate engine	Daily job that checks every active record against its deadline and grace period then applies allowance suspend and recover logic, as well as candidacy termination logic

Component	Responsibility
Reminder engine	Sends automated emails for missing documents, corrections, grace-period warnings, and terminations
Letter generator	Auto-fills PDF templates to produce Offer and Admission Letters the moment an approval happens
Eligibility reporting engine	Builds the consolidated allowance eligibility list from stage gate and payment data
Conversational FAQ engine	Answers student questions on status, missing documents, and deadlines by feeding live data into an LLM API

3.4.1 Presentation Layer

The presentation layer is the web portal where all six user roles access and use the system. It is developed using HTML5, CSS3, Bootstrap 5, and JavaScript. This layer handles user login, session management, and displays the appropriate dashboard for each user role, as described in Chapter 4. Since all six roles use the same web portal, the system shows different menus, pages, forms, and dashboard information based on the logged-in user's role instead of creating a separate website for each role.

3.4.2 Logic and Processing Layer

The logic and processing layer contains the five main system functions introduced in Table 3, namely the workflow engine, stage gate engine, reminder engine, letter generator, and eligibility reporting engine. Each function is developed as a separate PHP service, making the system easier to manage and maintain. This allows changes to be made to one function, such as the suspend, terminate, or recover rules described in Section 3.6.2, without affecting other parts of the system, including the approval process. This separation helps reduce errors and makes future updates easier.

3.4.3 AI/Intelligence Layer

The AI layer is positioned between the logic and processing layer and the presentation layer. It provides the conversational chatbot described in Section 3.7 to help users answer common questions about the system. Instead of storing its own copy of student data, the chatbot retrieves information from the same MySQL database used by the other system functions. This ensures that the information provided by the chatbot is consistent with the data displayed on the system dashboards.

3.4.4 Data Storage Layer

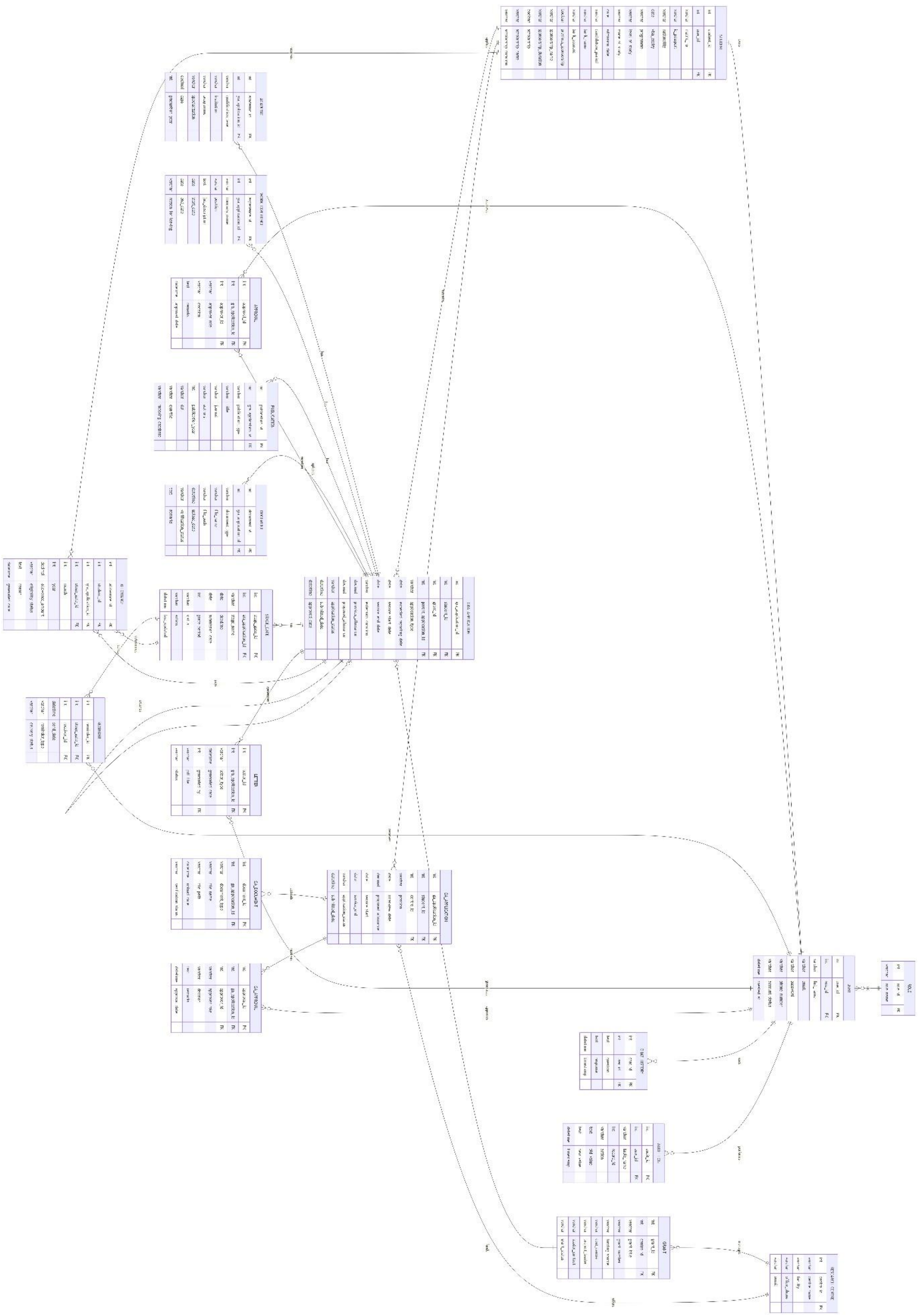
The data storage layer uses a single MySQL database to store all the relational tables described in Section 3.5. Using one central database allows all system modules to access the same data instead of keeping separate copies. As a result, the dashboards described in Chapter 4, the chatbot in Section 3.7, and the Allowance Eligibility List in Section 3.6.4 always display consistent and up-to-date information.

3.5 Database Design

The database is the single source of truth for the entire GRA/GA lifecycle. It is designed as a relational MySQL schema comprising more than twelve tables, grouped here into three categories:

- application and approval tables
- stage gate and allowance tables
- support tables covering documents, letters, and notification

For Clearer View, can press the link in the appendices



3.5.1 Core Database Table Schema

Table	Purpose	Key Fields
role	Stores the user roles used by the system, including Student, Supervisor, Research Centre, Admin, GRS Executive, and Senior Director.	role_id (PK), role_name
user	Stores user account information, login credentials, and role assignment for all system users.	user_id (PK), role_id (FK), full_name, email, password, phone_number, account_status, created
student	Stores student profile information required for GRA and GA applications.	student_id (PK), user_id (FK), matric_no, programme, level_of_study, mode_of_study, admission_date, candidature_period, nationality, bank_name, bank_account
research_centre	Stores information about research centres responsible for research grants and GA interviews.	centre_id (PK), centre_name, faculty, office_phone, email
grant	Stores research grant information associated with GRA applications.	grant_id (PK), centre_id (FK), grant_title, grant_number, funding_source, cost_centre, project_leader, leader_contact, grant_status
gra_application	Stores Graduate Research Assistant (GRA) application details submitted by students, including new and extension applications.	gra_application_id (PK), student_id (FK), grant_id (FK), parent_application_id (FK), application_type, expected_reporting_date, service_start_date, service_end_date, proposed_allowance, application_status
academic	Stores the applicant's academic qualification details	academic_id (PK), gra_application_id (FK), qualification_level, institution, programme, specialization, cgpa, graduation_year
work_experience	Stores the applicant's previous working experience.	academic_id (PK), gra_application_id (FK), qualification_level, institution, programme, specialization, cgpa, graduation_year

Table	Purpose	Key Fields
publication	Stores publication records submitted by applicants.	qualification_level, institution, programme, specialization, cgpa, graduation_year
document	Stores metadata and verification status for all uploaded supporting documents.	document_id (PK), gra_application_id (FK), document_type, file_name, file_path, upload_date, verification_status
approval	Stores approval records throughout the GRA workflow.	approval_id (PK), gra_application_id (FK), approver_id (FK), approver_role, decision, approval_date
stage_gate	Stores stage gate milestones, deadlines, grace periods, and	stage_gate_id (PK), gra_application_id (FK), stage_name, deadline, submission_date, grace_period, status, action
reminder	Stores reminder notifications generated for upcoming or overdue stage gates.	reminder_id (PK), stage_gate_id (FK), receiver_id (FK), send_date, reminder
letter	Stores system-generated letters such as approval, extension, and termination letters.	letter_id (PK), gra_application_id (FK), letter_type, generated_date, generated_by (FK), pdf_file, status
ga_application	Stores Graduate Assistant (GA) application details submitted by students.	letter_id (PK), gra_application_id (FK), letter_type, generated_date, generated_by (FK), pdf_file, status
ga_document	Stores supporting documents uploaded for GA applications.	document_id (PK), ga_application_id (FK), document_type, file_name, file_path, upload_date, verification_status
ga_approval	Stores approval decisions for GA applications.	approval_id (PK), ga_application_id (FK), approver_id (FK), approver_role, decision, approval_date
allowance	Stores monthly allowance eligibility and payment records for active GRA students.	allowance_id (PK), student_id (FK), gra_application_id (FK), stage_gate_id (FK), month, year, allowance_amount, eligibility_status, generated_date

chat_history	Stores chatbot conversations between users and the AI assistant.	chat_id (PK), user_id (FK), question, response, timestamp
Table	**Purpose**	**Key Fields**
audit_log	Stores user activities and system actions for auditing and traceability purposes.	audit_id (PK), user_id (FK), table_name, record_id, action, timestamp

3.5.2 Additional Data Attributes Required for Timeline Monitoring and Analytics

Several of the analytical dashboards presented in Chapter 4 require more information than the current status of an application or GRA record. Although the core database schema described in Section 3.5.1 is sufficient to support the system's operational functions, additional historical information must also be stored to support monitoring, reporting, and data analysis. These historical records allow the system to monitor how an application progresses throughout its lifecycle instead of only showing its latest status.

For example, once an application moves from *Pending* to *Approved*, the previous status alone is no longer enough to determine how long the application remained under review or which user was responsible for processing it. Therefore, important workflow events are recorded whenever they occur so that the complete history of each application can be analysed later. This historical information enables the system to generate meaningful dashboards, identify workflow bottlenecks, monitor processing performance, and provide management with useful insights for decision-making.

Table 5 summarises the additional data attributes that are stored in the database to support these analytical functions.

Table 5: Additional Data Attributes for Monitoring and Analytics

Data Attribute	Captured In	Purpose for Monitoring and Analytics
Stage entry timestamp and stage exit timestamp	**stage_gate**	Records the time a student enters and completes each stage gate. These timestamps are used to calculate the duration spent at each stage and to identify workflow delays and bottlenecks.
Approval decision timestamp and approver information	**approval**	Stores the approval date, approver role, and decision for every approval activity. This information is used to analyse approval turnaround time and workload across different user roles.

Grace period start date and grace period end date	**stage_gate**	Records the beginning and end of the grace period after a missed milestone. These dates allow the system to display stage gate timelines and determine Whether a student remains eligible, is suspended, or is terminated
Suspension or termination status and effective date	**allowance**	Records any changes to a student's allowance eligibility due to missed milestones or administrative decisions. This information supports allowance monitoring and suspension or termination trend analysis.
Monthly allowance payment status and payment amount	**allowance**	Stores the monthly allowance payment history, including payment amount and eligibility status. These records are used to generate the Allowance Eligibility List and monthly payment reports.
Document submission and verification status	**document**	Records the upload date, verification status, and completeness of every required supporting document. This information is used to monitor document submission progress and document verification status.
Programme level (Master or PhD)	**student**	Stores the student's programme level to support comparisons between Master's and PhD students in dashboard reports, including stage gate progress and allowance statistics.
Reminder sent date and delivery status	**reminder**	Records the date reminders are sent and whether they have been delivered successfully. This information supports reminder monitoring and future analysis of reminder effectiveness.
Letter generation date and letter type	**letter**	Records every system-generated letter, such as approval, extension, or termination letters. These records allow the system to monitor document generation activities.
Application submission and approval dates	**gra_application** and **ga_application**	Stores the submission and approval timeline for both GRA and GA applications. These dates are used to measure overall application processing time and workflow performance.

The system records these attributes at the time each workflow event occurs instead of updating only the latest record. This approach is important because many of the analytical dashboards described in Chapter 4 depend on historical information rather than the application's current status alone. For example, reports that measure approval duration, workflow delays, stage gate compliance, allowance eligibility, and document completion require previous workflow activities to remain available for analysis.

For this reason, the workflow engine records every significant workflow event, including application submission, approval decisions, stage gate progression, reminder generation, document verification, allowance status updates, and letter generation. Instead of replacing previous information, the system preserves these records as historical data. This approach provides a complete record of each application's lifecycle, improves traceability, and supports accurate reporting, dashboard visualisation, and future performance analysis.

3.6 Process Flow of the Three Modules

The system is organised around three functional modules, each of which is described below as a step-by-step process flow.

3.6.1 Module 1: GRA Application

Actor	Action
Student	Applies for GRA, accepts the terms and conditions, completes Sections B–F, uploads the required supporting documents, and submits the application.
Admin / GRS Exec	Verifies the submitted application and supporting documents. If any required document is missing or incorrect, the system automatically sends a reminder notification requesting the student to update and resubmit the application.
Supervisor	Reviews the application and marks the approval status as **"In Process"** while confirming the proposed appointment details.
Supervisor	Updates the approval status to **"Settled"** and records the approved service duration before forwarding the application for final approval.
Senior Director	Reviews the completed application and approves or rejects the appointment.
System	Automatically creates the GRA record, generates the Stage Gate schedule, and produces the Offer Letter and Admission Letter for approved applicants.
Student	Downloads the generated letters and begins the Stage Gate monitoring process. When the approved appointment period is

Actor	Action
	nearing its end, the student may submit a **GRA Extension** application, which follows the same approval workflow before the service period is extended.

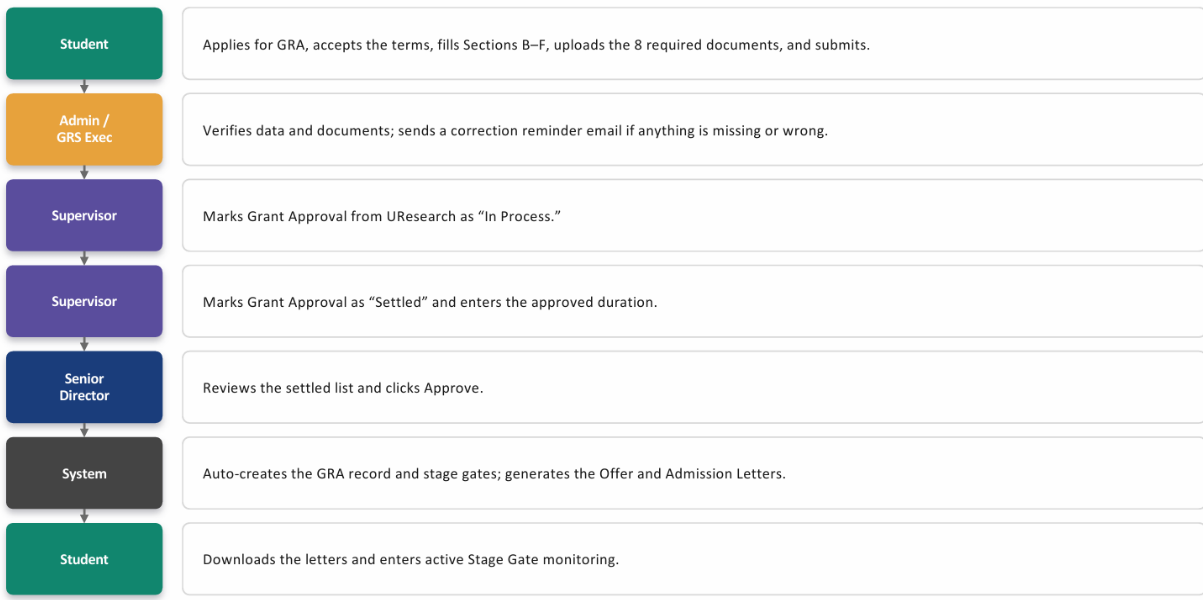


Figure 5: Process Flow Diagram – Module 1 GRA Application

Two stages in this workflow deserve special attention because they were identified as the main sources of delay during the requirement gathering phase discussed in Section 3.8.1. The first is the **Admin/GRS Executive verification stage**, where submitted applications are reviewed to ensure that all required supporting documents have been uploaded correctly. Under the current manual process, if any document is missing or incomplete, the application is returned to the student via email, which often delays the application while waiting for staff to manually identify and communicate the issue. In the proposed system, this process is improved through the **Reminder Engine**, which automatically notifies the student whenever a required document is missing or fails verification. This allows students to correct and resubmit the documents promptly, reducing unnecessary delays in the application process.

The second delay point occurs during the **Supervisor approval stage**. To better reflect the actual approval process, the proposed system uses two approval statuses: **"In Process"** and **"Settled"**. The **"In Process"** status indicates that the Supervisor is reviewing the application and confirming details such as the proposed service duration or allowance. Once all information has been verified and finalised, the status is updated to **"Settled"**, allowing the application to proceed to the next stage of the workflow. Maintaining these two approval statuses provides a clearer view of the application's progress and enables students, Supervisors, and administrators to monitor the approval process more effectively.

3.6.2 Module 1: Stage Gate Monitoring

Once a GRA record has been created, it enters continuous stage gate monitoring. The Research Proposal Defence (RPD), which forms Stage 1, behaves differently from every stage that follows it, because a student who misses the RPD deadline can still recover their allowance once the milestone is eventually completed, whereas a student who misses a later stage is terminated outright with no possibility of recovery.

RPD – Stage 1 (Recoverable)	Later Stages 2, 3, and 4 (PHD only) (Not Recoverable)
Deadline reached, RPD not yet completed	Deadline reached, milestone not yet completed
Grace period: monthly reminder – "finish or lose allowance"	Grace period: monthly reminder – "finish or get terminated"
Grace period ends, still not done → allowance suspended (one-time notice)	Grace period ends, still not done → student auto-terminated (one-time notice)
RPD eventually completed → allowance back-paid from the month after grace ended	No recovery – termination is final and permanent

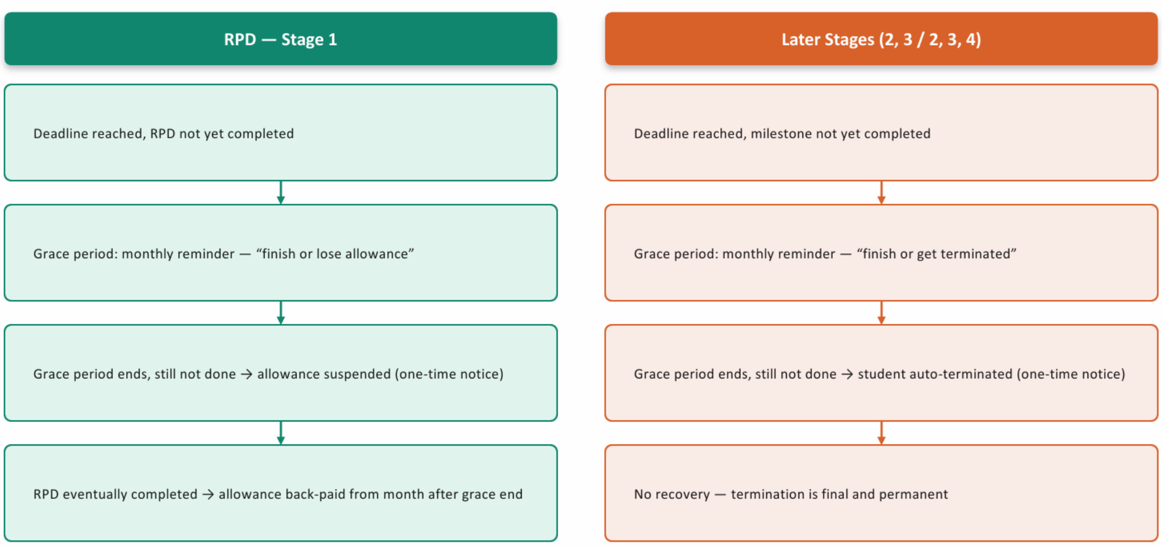


Figure 6: Stage Gate Monitoring Decision Flow

Master

Stage Gate Level	Requirement for GA/GRA continuation	Deadline to complete	Grace Period
1	RPD Completion	Month 3	Month 3 – Month 6
2	One (1) article accepted in journals indexed by WoS/ERA or Scopus	Month 12	Month 12 – Month 18
3	One (1) article published in journals indexed by WoS/ERA or Scopus	Month 18	Month 18 – Month 24

Stage gate of master student (GRA)

PhD

Stage gate level	Requirement for GA/GRA continuation	Deadline to complete	Grace Period
1	RPD Completion	Month 6	Month 6 – Month 9
2	One (1) journal article accepted	Month 18	Month 18 – Month 30
3	One (1) journal article published in Q1/Q2 OR One (1) journal article accepted & One (1) article published in journals indexed by WoS/ERA or Scopus	Month 30	Month 30 – Month 36
4	Soft bound submission for RCS	Month 36	Month 36 – Month 42

Stage gate of phd student (GRA)

3.6.3 Module 2: GA Application

Actor	Action
Student	Applies for the Graduate Assistant (GA) position, completes the application form, and submits the required information.
Admin / CGS	Reviews the application to determine whether the student meets the eligibility requirements. If the student is not eligible, the application

Actor	Action
	status is updated to **Rejected**, the student is notified, and the process ends.
Admin / CGS	Forwards eligible applications to the relevant **Research Centre** based on the student's supervisor or assigned research centre.
Research Centre	Conducts the interview and submits the result as Pass or Fail, with supporting notes.
Admin / CGS	Reviews the interview outcome, records the final decision, and approves or rejects the application in the system.
System	Auto-generates the GA Offer Letter and Program Offer Letter and creates the GA record.
Student	Downloads the letters and enters Stage Gate monitoring, shared with GRA.

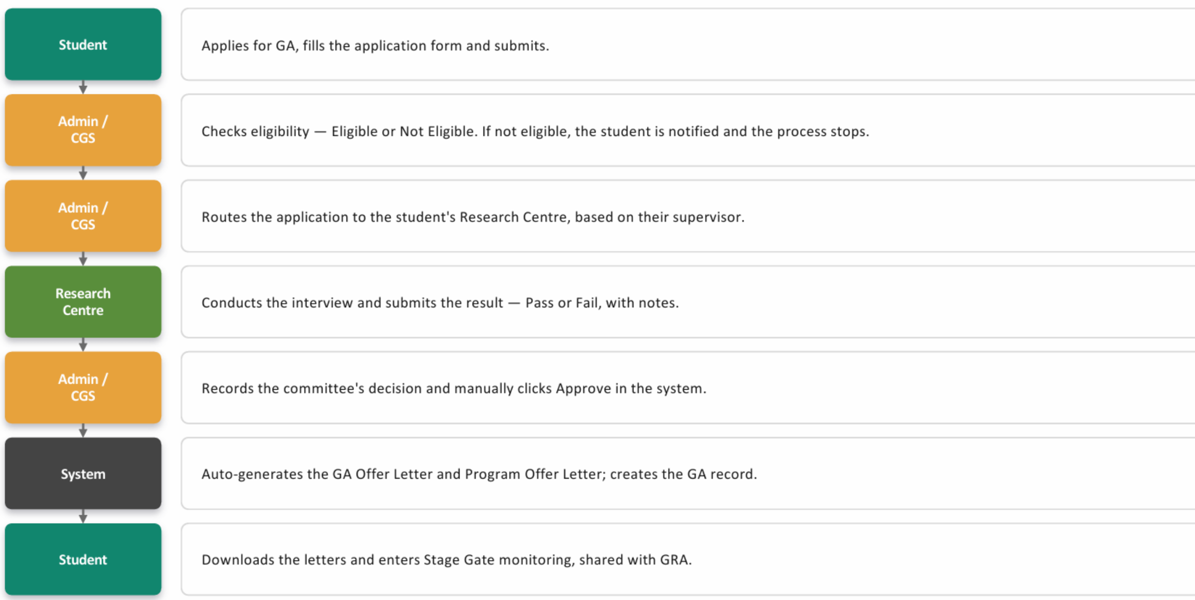


Figure 7: Process Flow Diagram – Module 2 GA Application

Module 2 differs slightly from Module 1 because it includes an interview stage conducted by the student's **Research Centre** as part of the application process. This is because Graduate Assistant (GA) positions usually involve teaching, laboratory, or administrative responsibilities within a specific research centre. Therefore, the Research Centre is responsible for assessing whether the applicant has the appropriate skills and is suitable for the position.

After a student submits a GA application, the system first verifies that the application meets the required eligibility criteria. Once the application is verified, it is forwarded to the relevant Research

Centre to arrange an interview. After the interview has been completed, the result is recorded in the system, and the application proceeds to the approval stage.

Throughout this process, the eligibility verification, interview outcome, and approval decision serve as key decision points in the workflow. If the applicant does not meet the eligibility requirements, fails the interview, or the application is not approved, the application status is updated accordingly, and the student is automatically notified of the outcome. Otherwise, the application is marked as **Approved**. This workflow ensures that every application reaches a clear outcome while making the process easier for students, Research Centre staff, and administrators to monitor.

3.6.4 Module 3: Allowance Eligibility

Unlike Modules 1 and 2, **Module 3** is not designed for students to submit applications or complete workflow activities. Instead, it serves as a reporting module that displays the latest allowance eligibility status of active GRA students using data already stored in the system. Rather than requiring users to enter new information, the module retrieves and processes data from the **GRA application**, **approval**, **stage gate**, and **allowance** records to determine each student's current eligibility status.

To ensure that the information remains accurate and up to date, the system automatically generates the **Allowance Eligibility List** through a scheduled process that runs once every day. During this process, the system checks each student's current stage gate progress, approval records, and allowance eligibility before updating the report. The generated report can be accessed by **Admin**, **GRS Executive**, **Supervisor**, and **Senior Director** according to their respective access permissions. This automation eliminates the need for staff to manually compile allowance information while ensuring that the latest eligibility records are always available.

Actor	Action
System	Daily job evaluates every active GRA/GA record's current stage gate status.
Stage Gate Engine	Applies RPD suspend/recover logic, or later-stage terminate logic, per student.
System	Updates allowance_payment_log for the month as paid or not paid, with the corresponding amount.
System	Builds a consolidated Allowance Eligibility List, filterable by status and level.

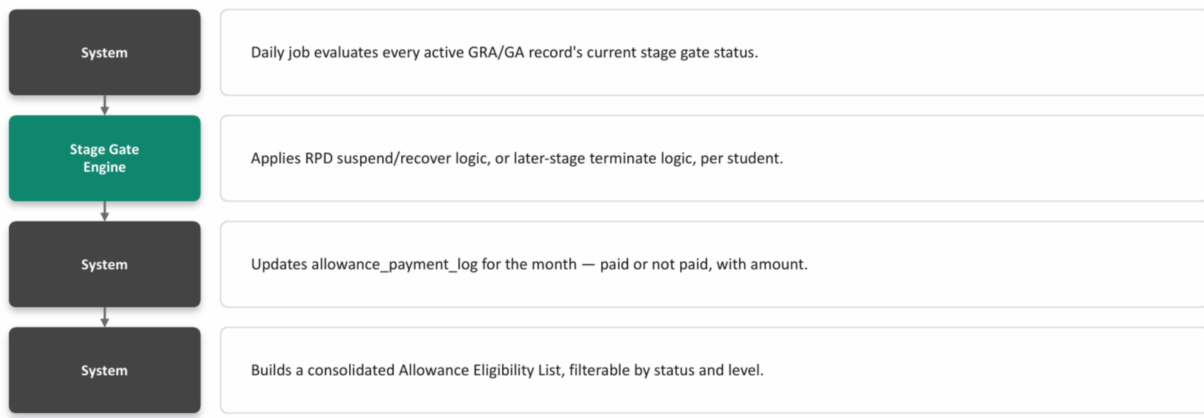


Figure 8: Process Flow Diagram – Module 3 Allowance Eligibility

The **Allowance Eligibility List** can be viewed by **Admin**, **GRS Executive**, **Supervisor**, and **Senior Director** according to their assigned roles and responsibilities. The information is retrieved from the **allowance** table, which records each student's monthly allowance eligibility and payment status. These historical records not only support the generation of the Allowance Eligibility List but also provide the data required for the analytical dashboards discussed in Chapter 4.

3.6.5 Cross-Module Data Flow

Although Modules 1, 2, and 3 are described separately, they are designed to work together within the same integrated system. All three modules share the same student database and user management system, allowing information to be accessed consistently across different functions. However, only students who are successfully approved through **Module 1 (GRA)** proceed to the **Stage Gate** monitoring process described in Section 3.6.2. The stage gate records are then used by the system to determine each student's monthly allowance eligibility, which is displayed in **Module 3** without requiring any additional data entry from staff.

This integrated design allows Module 3 to function solely as a reporting module. Since all the information required for allowance eligibility is already captured during the GRA application, approval, and stage gate processes, there is no need for students or administrators to submit a separate allowance application. This reduces manual work, improves data consistency, and ensures that the Allowance Eligibility List is always generated using the latest information stored in the database.

3.7 Artificial Intelligence Integration

The system's AI layer includes a conversational FAQ chatbot that allows students to obtain information about their application status without having to contact CGS staff. Instead of relying on a fixed list of pre-written responses, the chatbot retrieves the student's latest information directly from the system database whenever a question is submitted. This information is then sent to the Large Language Model (LLM) API together with the student's question, enabling the chatbot to generate responses that are both natural and based on the student's current records. As a result, students can receive accurate and up-to-date information quickly, reducing the need for manual enquiries and improving the overall user experience.

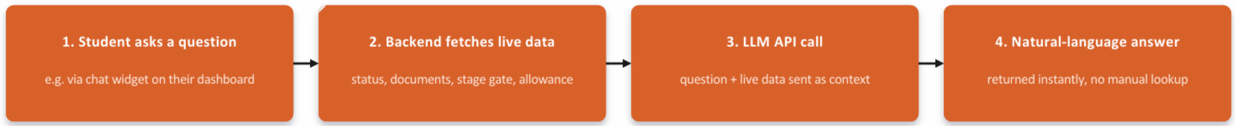


Figure 9: AI Chatbot Interaction Flow

Example Student Question	Data Source Queried
"What's my status?"	applications, grant_approvals status fields
"What am I missing?"	documents table – lists which of the eight required documents are outstanding
"When's my next deadline?"	stage_gates – current stage deadline
"Am I getting paid this month?"	allowance_payment_log

This approach keeps the AI component practical and suitable for the scope of this final year project. Instead of developing and training a custom AI model, the system uses a predefined prompt template, retrieves the relevant student information from the database, and sends the request to an LLM API to generate a response. This simplifies the implementation while still allowing the chatbot to provide accurate and context-aware answers. For this reason, the AI functionality described in Section 1.4.3 is intentionally limited to a conversational chatbot rather than more advanced features, such as predictive analytics or machine learning models, which are beyond the scope of this project.

3.8 Project Timeline

The project is planned across two semesters, FYP 1 and FYP 2, each spanning fourteen weeks. The Gantt charts below set out the activities, milestones, and their scheduled weeks for each semester.

3.8.1 FYP 1 Gantt Chart and Milestones

Activity / Week	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Project Title Confirmation & Form 1A Submission	■													
Preliminary Research & Literature Review	■	■	■											
Requirement Gathering: Modules 1, 2 & 3 Process Mapping		■	■	■										
Identify Interactive Waterfall Methodology & Tools			■	■										

Activity / Week	1	2	3	4	5	6	7	8	9	10	11	12	13	14
System Design: Database Schema, Architecture & AI Layer				■	■	■								
Proposal Defence Preparation					■	■								
Proposal Defence							■							
FYP1 Interim Report Preparation							■	■	■	■	■	■		
UI/UX Mockup & Wireframe Design (6 role dashboards)						■	■	■	■					
Submission of FYP1 Interim Report Draft to Supervisor											■			
Submission of Final FYP1 Interim Report													■	
Baseline Environment Setup (XAMPP, PHP8, MySQL schema)										■	■	■	■	■

Table 11: FYP 1 Gantt Chart and Milestones

3.8.2 FYP 2 Gantt Chart and Milestones

Activity / Week	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Database & Schema Finalization (all 3 modules)	■	■												
Module 1: GRA Application, Approval Workflow, Document Upload	■	■	■	■										
Module 1: Stage Gate & Notification Engine			■	■	■									
Submission of Progress Assessment 1 Deliverables					■									
Module 2: GA Application, Eligibility Check, Interview Workflow					■	■	■							
Module 3: Allowance Eligibility Dashboard & Reports							■	■	■					
AI Layer: Chatbot & Predictive Risk Model Integration								■	■	■				
System Integration (Auth, PHPMailer, Dompdf, LLM API)									■	■				
Submission of Draft Dissertation & Draft Technical Paper										■				

Activity / Week	1	2	3	4	5	6	7	8	9	10	11	12	13	14
System Testing & UAT (role-based scenarios)														
Submission of Dissertation (Soft Bound) & Technical Paper														
Project VIVA Strategy & Slides Preparation														
Final Project Presentation & VIVA Defense														
Post-VIVA Corrections & Documentation Polish														
Submission of Project Dissertation (Hard Bound)														

Table 12: FYP 2 Gantt Chart and Milestones

As shown in the two Gantt charts, **FYP 1** focuses mainly on project planning, requirement gathering, system design, and the proposal defence. In contrast, **FYP 2** focuses on system development, module integration, testing, and the preparation of the final dissertation and VIVA presentation. The system development environment is prepared during the final stage of **FYP 1** while report writing is still in progress. This allows development activities to begin immediately at the start of **FYP 2** without spending additional time on software installation and environment configuration.

3.9 Security and Access Control

Since the system stores students' personal information, supporting documents, and allowance records, security and access control are important considerations throughout the system design. To protect sensitive information, every user is required to log in before accessing the system. User authentication is implemented using PHP session-based authentication, while passwords are stored as hashed values instead of plain text to improve account security.

The system also implements **Role-Based Access Control (RBAC)** to ensure that users can only access the functions and information relevant to their responsibilities. For example, Supervisors can only view and manage applications, stage gate records, and allowance information for students under their supervision, whereas **GRS Executive** and **Senior Director** users have broader access to monitor and manage records across the system.

The main security measures implemented in the system are summarised below.

- **Authentication:** Every user must log in using a valid email address and password. PHP sessions are used to maintain user authentication throughout the session, and inactive sessions are automatically terminated after a predefined period to reduce the risk of unauthorised access.

- **Authorisation:** Access to dashboards, forms, and system functions is controlled using Role-Based Access Control (RBAC). Each user is only allowed to access the features and data associated with their assigned role, following the six user roles defined in Section 4.1.
- **Audit Logging:** All important actions that change the status of an application, approval, stage gate, or allowance record are recorded in the **audit_log** table. Each log entry stores the user responsible for the action together with the date and time it was performed, providing traceability and accountability throughout the system.
- **Document Security:** Supporting documents uploaded by students are stored outside the public web directory and can only be accessed through the application after the user's permissions have been verified. This prevents direct access to uploaded files through their file location and helps protect confidential student documents.

3.10 Non-Functional Requirements

In addition to the functional requirements already described through the process flows in Section 3.6, the system is also expected to satisfy a set of non-functional requirements, summarized in the table below.

Requirement	Description
Performance	The system should respond quickly when users access dashboards, submit applications, or retrieve records. Under normal local deployment conditions, pages should load and refresh within a few seconds to provide a smooth user experience.
Usability	The system should provide a simple and user-friendly interface that can be used by students, Supervisors, Research Centre staff, Admin, GRS Executive, and Senior Director with minimal training. Clear navigation and consistent layouts should help users complete their tasks efficiently.
Reliability	The system should consistently perform key automated processes, including Stage Gate monitoring, reminder generation, allowance eligibility updates, and letter generation, without missing any eligible student records. This helps ensure that application status and allowance information remain accurate.
Maintainability	The system should be developed using a modular architecture, with separate modules for GRA Management, GA Management, Allowance Eligibility, and the AI chatbot. This structure makes future maintenance, debugging, and feature enhancements easier without affecting other modules.
Scalability	The database and system architecture should support an increasing number of students, applications, and uploaded documents without

	requiring major changes to the core database structure or application workflow.

CHAPTER 4: SYSTEM DESIGN AND EXPECTED RESULTS

4.1 User Roles Overview

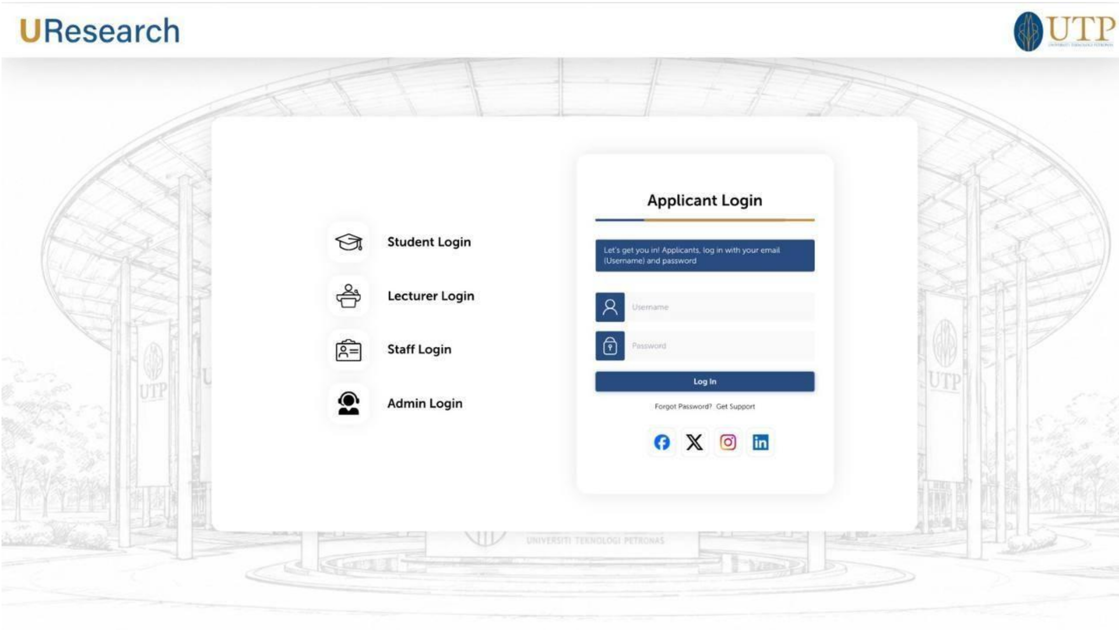
The system defines six distinct user roles, each with a different scope of visibility and a different set of actions available to them. Table 13 summarizes each role's core responsibility within the system.

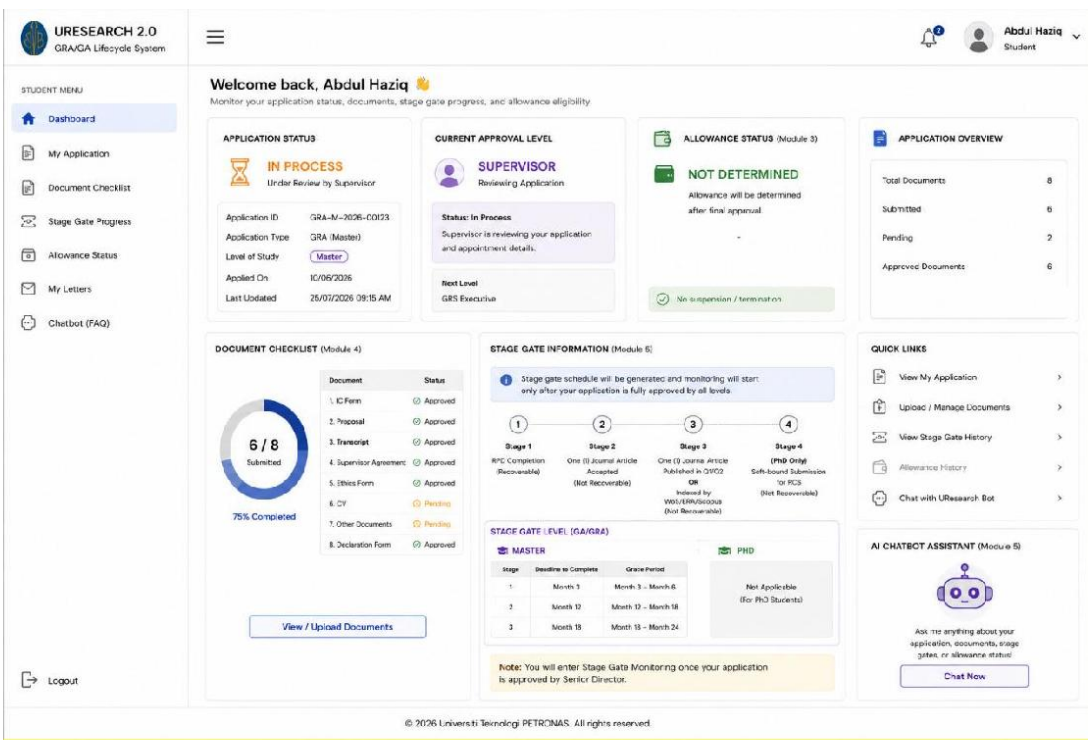
Role	Core Responsibility Within the System
Student	Submits GRA/GA applications, uploads supporting documents, tracks their own stage gate and allowance status, and interacts with the chatbot.
Admin (CGS)	Verifies application data and documents, checks GA eligibility, records committee decisions, and manages the overall application queue.
GRS Exec	Supports Admin in verification and monitoring, and views consolidated dashboards across all students.
Supervisor	Marks Grant Approval status for their supervised students and monitors their stage gate progress.
Senior Director	Gives final approval on settled Grant Approvals and monitors system-wide trends across all research centres.
Research Centre	Conducts GA interviews and submits Pass/Fail results with supporting notes.

4.2 Role-Based Dashboard Design

Each of the six roles is given a dashboard tailored to the decisions they are responsible for. The student dashboard is designed for self-monitoring, while the remaining five roles are given progressively wider, more aggregated views suited to oversight and escalation.

User authentication page:





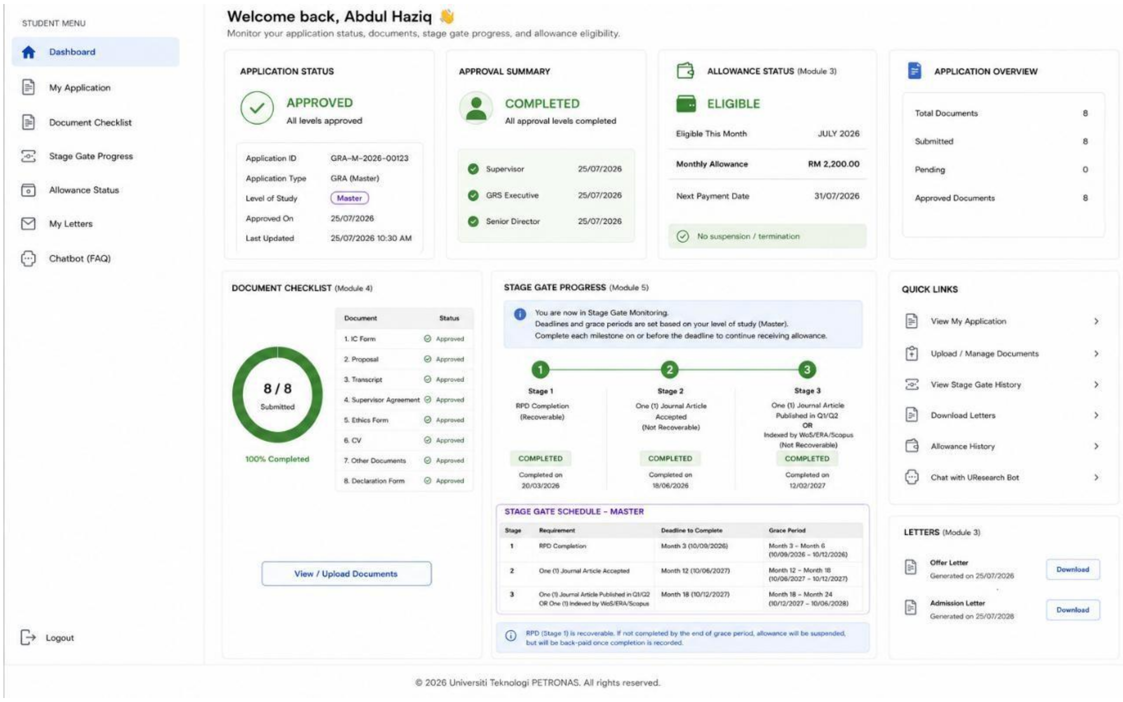


Figure 10: Student Dashboard Wireframe

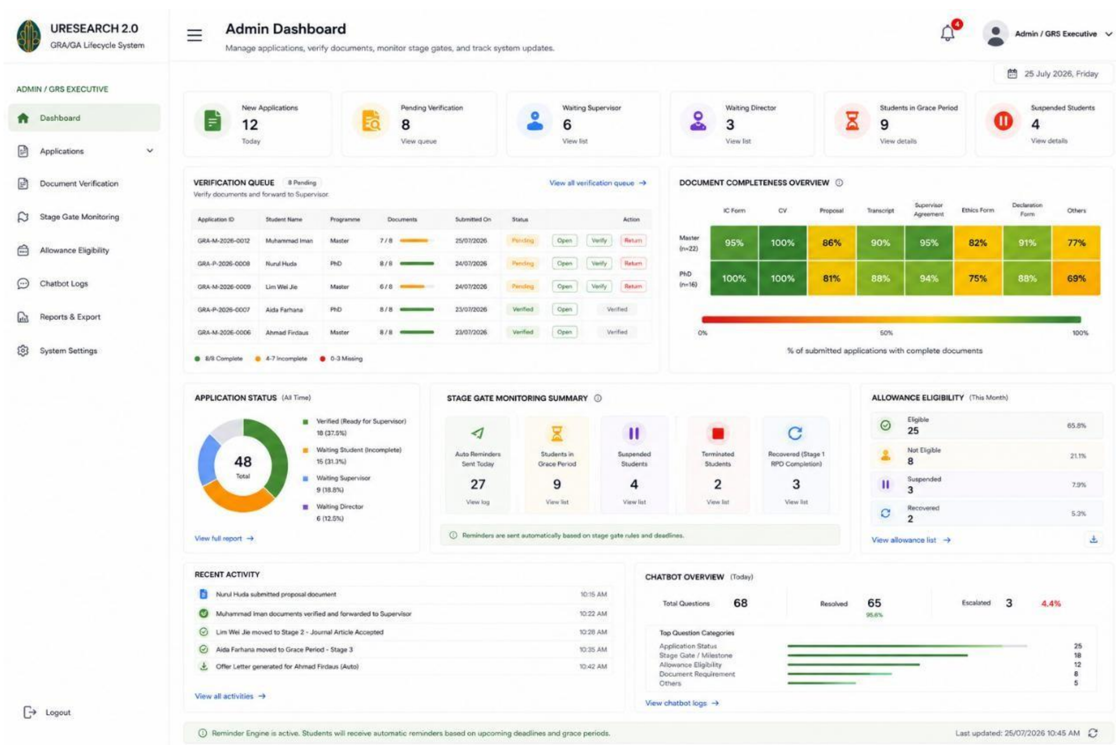


Figure 11: Admin/GRS Exec Dashboard Wireframe

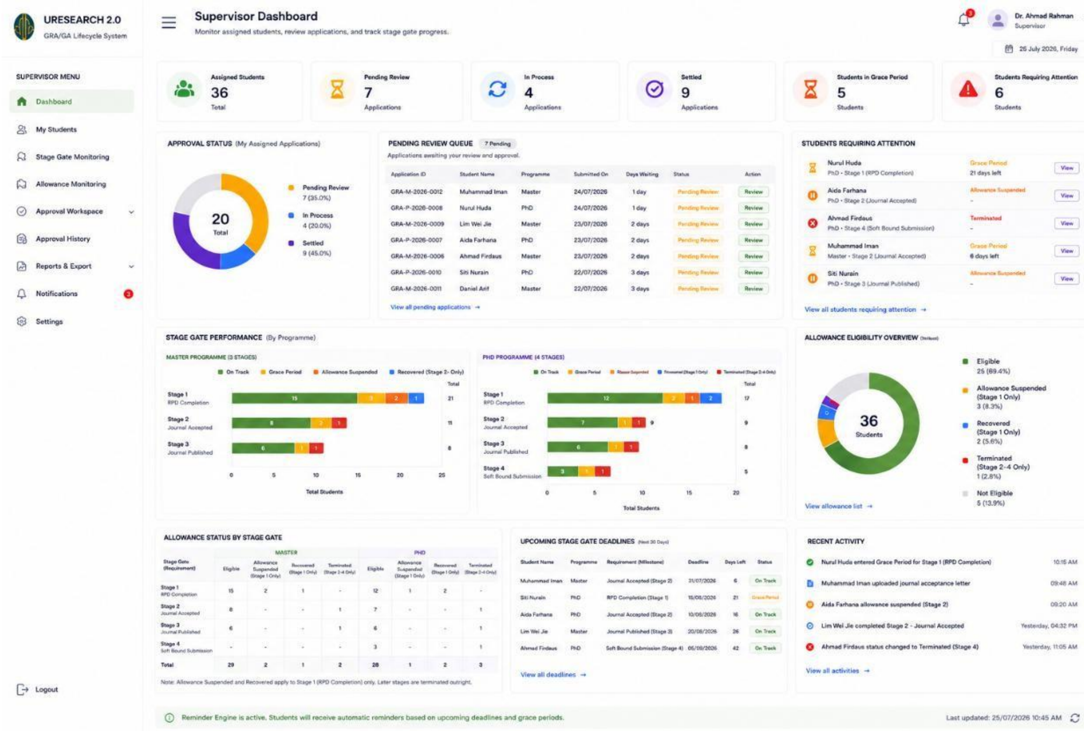


Figure 12: Supervisor Dashboard Wireframe

Figure 13: Senior Director Dashboard Wireframe

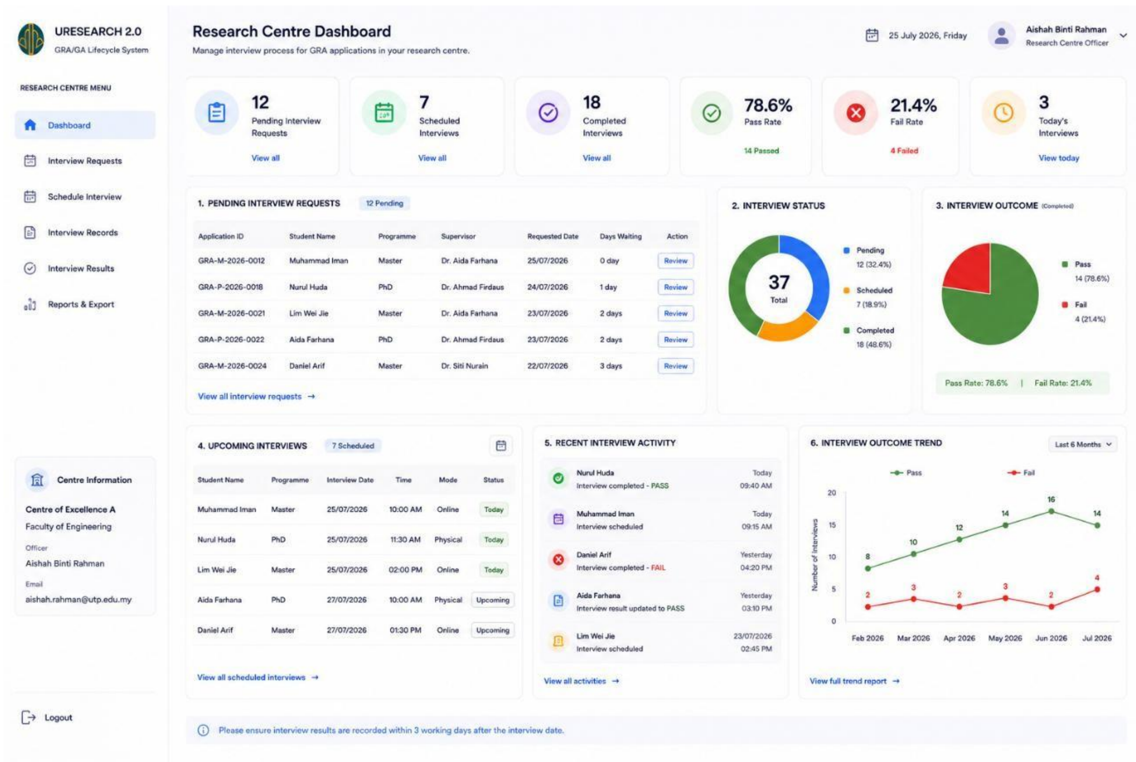


Figure 14: Research Centre Dashboard Wireframe

The **Student Dashboard** is designed to provide students with a clear overview of their application and study progress. The dashboard allows students to quickly check their application status, identify any outstanding supporting documents, and view their current allowance eligibility. In addition, the AI chatbot described in Section 3.7 is integrated into the dashboard, allowing students to ask questions about their application or GRA status without needing to contact CGS staff. This provides students with quick access to information while improving the overall user experience.

The **Admin**, **GRS Executive**, and **Supervisor** dashboards are designed using the same set of dashboard components, including summary charts, application statistics, overdue Stage Gate records, and progress visualisations. Although the layout is similar for these three user roles, the information displayed depends on the user's access permissions. **Admin** and **GRS Executive** users are able to view records for all students in the system, while **Supervisors** can only access information related to students under their supervision. Using a common dashboard design across these roles provides a consistent user experience while allowing the system to display different data based on user responsibilities.

The **Senior Director Dashboard** provides a high-level overview of the overall performance of the GRA and GA management process. Instead of displaying individual student records, the dashboard focuses on summary information such as application statistics, approval progress, and processing delays across different approval stages. These visualisations are generated using historical data stored in the system and help management identify workflow bottlenecks, monitor overall performance, and support decision-making at the university level.

Role	Dashboard Content
Student	Personal application status card, supporting document progress ring (e.g., **6 of 8 documents submitted**), Stage Gate timeline, current allowance eligibility status with the reason if suspended, AI chatbot for application enquiries, and quick links to application details and generated letters.
Admin (CGS) / GRS Exec	Approved versus pending application donut chart, Stage Gate funnel, overdue Stage Gate list sorted by overdue days, document completeness heatmap, searchable student list, and filters by application status, programme level (Master/PhD), Research Centre, Supervisor, and application period.
Supervisor	The same dashboard components as the Admin and GRS Executive, limited to students under the Supervisor's supervision, with filters by application status, Stage Gate, programme level, and application period.
Senior Director	System-wide approved versus pending application chart, Stage Gate funnel grouped by Master's and PhD students, approval delay chart, allowance suspension and termination trend, KPI summary cards, and filters by faculty, Research Centre, programme level, application period, and approval status.
Research Centre	Pending interview request queue, interview schedule, average interview turnaround time, Pass/Fail ratio chart, searchable applicant list, and filters by interview status, programme level, application period, and Supervisor.

All dashboards provide common data management features to improve usability, including search functionality, sorting, filtering, and pagination where appropriate. The available filter options depend on the user's role and access permissions. For example, Admin and GRS Executive users can filter records by programme level, Research Centre, Supervisor, application status, and date range, while Supervisors can only filter records related to students under their supervision. These features allow users to locate and analyse relevant information more efficiently, particularly when managing a large number of student records.

4.3 Data Required for Monitoring and Analytics

The effectiveness of the dashboards presented in the previous section depends on the availability of accurate and up-to-date data. Each dashboard visualisation retrieves information directly from the system database, allowing users to monitor application progress, student status, and overall workflow performance in real time. To support these visualisations, the database has been designed to capture the key information required by each dashboard component.

- **Application summary charts** (such as approved versus pending applications) retrieve data from the **gra_application**, **ga_application**, and **approval** tables. These charts rely on the application status and approval records to provide an overview of the current application progress.
- **Stage Gate progress charts**, **overdue milestone lists**, and **processing delay reports** use information stored in the **stage_gate** table. The system records each milestone's deadline, submission date, and completion status, allowing overdue days and processing duration to be calculated automatically without storing these values separately.
- **Supporting document progress indicators** and **document verification dashboards** obtain their information from the **document** table. Each uploaded document stores its upload date and verification status, enabling the system to identify missing, pending, or verified documents for every application.
- **Allowance eligibility indicators** and the **Allowance Eligibility List** retrieve data from the **allowance** table. Monthly allowance eligibility and payment records are used to display each student's current allowance status and generate historical allowance reports.
- **Research Centre dashboards** use data from the **ga_application** and **ga_approval** tables to monitor pending interviews, interview outcomes, and the overall progress of Graduate Assistant (GA) applications.
- **Management dashboards** combine information from the **gra_application**, **approval**, **stage_gate**, and **allowance** tables to generate summary statistics, approval trends, Stage Gate progress, and workflow performance indicators for supervisors and university management.

By designing the database to capture these data from the beginning of the project, every dashboard described in Section 4.2 can retrieve the required information directly from the database. This eliminates the need for manual data compilation, improves data consistency, and ensures that the dashboards always display the latest information available in the system.

4.4 Expected Outcomes

The expected outcomes of this project are used to evaluate whether the proposed system has successfully achieved its objectives. Each outcome is supported by a measurable criterion that can be verified during system testing and evaluation.

Expected Outcome	Measurement Criteria
Functional integrated platform	The system successfully supports all three modules, with each user role able to perform its assigned functions. All workflow, approval, Stage Gate, and allowance test cases are completed successfully.
Reduced manual processing	The system automates application routing, reminder notifications, Stage Gate monitoring, and letter generation, reducing the number of manual tasks previously performed by CGS staff.
Improved process transparency	Students and staff are able to monitor application progress, approval status, Stage Gate milestones, and allowance eligibility through the dashboards. System usability is evaluated using the System Usability Scale (SUS) questionnaire completed by selected users.
Accurate stage gate and allowance logic	The system correctly implements the Stage Gate rules, including milestone deadlines, grace periods, allowance suspension, termination, and recovery scenarios. All predefined test cases produce the expected results.
System performance	The system responds efficiently during normal operation, with dashboards, application forms, and reports loading within a reasonable time under the local deployment environment.
Reliable AI-driven insights	The chatbot retrieves the latest student information from the database and provides accurate responses to common application and allowance-related enquiries during user testing.

4.5 Testing Strategy

System testing is carried out in three stages, following the **Testing** phase of the Interactive Waterfall model described in Section 3.2. Each stage focuses on a different aspect of the system to ensure that all modules, workflows, and system functions operate correctly before deployment. The testing process begins with individual system functions, followed by interactions between modules, and finally evaluates the complete system from the users' perspective

Testing Level	Focus	Example Scenario
Unit Testing	Tests individual functions and modules independently.	Verify that the Stage Gate Engine correctly calculates the grace

Testing Level	Focus	Example Scenario
		period based on the milestone deadline and predefined grace period.
Integration Testing	Tests the interaction between different modules and shared system components.	Verify that an approved GRA application automatically generates the required letters, updates the application status, and sends a notification to the student.
User Acceptance Testing (UAT)	Evaluates the complete system based on real user tasks performed by representative users from each role.	Verify that when a Supervisor marks an application as **Settled**, the application appears correctly in the Senior Director's approval dashboard for final approval.

Particular attention is given to the **Stage Gate Engine**, as it contains the main business rules used to determine student progression and monthly allowance eligibility. Test cases are designed to verify different scenarios involving milestone deadlines, grace periods, allowance suspension, termination, and recovery. Each scenario is compared with the expected outcome to ensure that the system applies the correct workflow and allowance status under different conditions. Successful completion of these test cases provides confidence that the system functions as intended before deployment.

4.6 Risk Management

As with any system development project, several risks may affect the successful implementation of the proposed system. Identifying these risks during the development phase allows appropriate mitigation strategies to be planned and implemented before the system is deployed. Table X summarises the main risks identified for this project together with the corresponding mitigation measures.

Risk	Mitigation
Incorrect Stage Gate logic causes an incorrect allowance suspension or student termination.	Develop comprehensive test cases covering different Stage Gate scenarios, including milestone completion, grace periods, suspension, termination, and recovery. In addition, Admin users are provided with a manual override function to resolve exceptional cases.
The AI chatbot provides inaccurate or incomplete responses.	The chatbot retrieves the latest student information directly from the system database and includes this information when generating responses through the

Risk	Mitigation
	LLM API, reducing the likelihood of inaccurate answers.
Delays in system development due to the complexity of the workflow and Stage Gate modules.	Follow the Interactive Waterfall development model, allowing feedback from each phase to be incorporated before continuing to the next stage, thereby reducing the impact of design or implementation issues.
Unauthorised access to student information, supporting documents, or allowance records.	Implement user authentication, Role-Based Access Control (RBAC), password hashing, audit logging, and secure document storage as described in Section 3.9.
System errors caused by incorrect or incomplete user input.	Apply input validation, mandatory field checking, and document verification before allowing applications to proceed to the next stage of the workflow.

4.7 Alignment with Project Objectives

To close this chapter, Table 17 maps each of the three objectives stated in Section 1.3 back to the specific system features described across this chapter and Chapter 3, so that the connection between what the project set out to do and what has actually been designed is made explicit.

Objective (Section 1.3)	System Feature Delivering It
To design and develop a centralized online system for GRA/GA application, document submission, and multi-level approval.	Module 1 and Module 2 process flows (Sections 3.6.1 and 3.6.3), workflow engine, and document metadata storage (Section 3.5.1).
To implement an automated Stage Gate monitoring system for tracking research milestones, deadlines, and allowance status.	Stage gate engine and suspend/terminate/recover logic (Section 3.6.2), Module 3 allowance eligibility reporting (Section 3.6.4).
To provide role-based dashboards and AI-powered insights for real-time visibility across all stakeholders.	Role-based dashboard design for six roles (Section 4.2) and the conversational FAQ chatbot (Section 3.7).

CHAPTER 5: CONCLUSION AND FUTURE WORKS

5.1 Conclusion

The current GRA and GA management process at UTP relies heavily on manual handling, with information distributed across emails, spreadsheets, and paper-based records. As a result, stakeholders have limited visibility into application progress, research milestones, and allowance status, while missed deadlines may only be identified after they have already affected a student's progress.

This project addresses these challenges by designing and developing a centralized web-based system that integrates GRA and GA application management, Stage Gate monitoring, allowance management, and role-based dashboards into a single platform. The system also introduces workflow automation through application routing, document verification, reminder notifications, and automated letter generation, reducing the dependence on manual administrative processes.

One of the key contributions of the proposed system is the implementation of the Stage Gate Engine, which monitors research milestones, tracks deadlines, and applies the predefined allowance rules consistently. By embedding these rules directly into the system, the project helps reduce the risk of missed deadlines affecting student allowance eligibility while providing a more structured and transparent workflow for administrators and supervisors.

In addition, the AI chatbot enhances the user experience by allowing students to obtain information about their applications, Stage Gate progress, and allowance status through natural language questions. Because the chatbot retrieves information directly from the system database, its responses remain consistent with the information displayed on the dashboards.

Overall, the proposed system provides a more efficient, transparent, and centralized approach to managing the GRA and GA lifecycle. By integrating the key processes into a single platform, the system supports better communication between students, supervisors, CGS staff, and university management while improving the overall efficiency of graduate research administration.

5.2 Future Works

Since this interim report only covers the planning, requirement analysis, and design stages completed in FYP 1, the actual development of the GRA/GA Student Lifecycle Management System is planned to take place in FYP 2. The activities below outline what is expected to be carried out in the next semester, following the FYP 2 Gantt chart in Section 3.8.2.

- **Finalising the database and schema:** Before any module is built, the database design from Section 3.5 will be finalised across all three modules, so that any adjustments identified during early development do not require the schema to be redesigned partway through.

- **Building Module 1 (GRA Application):** The application form, document upload, and multi-level approval workflow (Admin → GRS Exec → Supervisor → Senior Director) described in Section 3.6.1 will be developed, along with the Stage Gate and notification engine described in Section 3.6.2, which handles deadline tracking, grace periods, and the suspend/terminate/recover logic.

- **Building Module 2 (GA Application):** The GA application, eligibility screening, Research Centre interview workflow, and committee approval process described in Section 3.6.3 will be developed next.

- **Building Module 3 (Allowance Eligibility):** The reporting module described in Section 3.6.4 will be built to consolidate stage gate and approval data into the Allowance Eligibility List, along with the dashboards and reports described in Section 4.2.

- **Integrating the AI layer:** The conversational FAQ chatbot described in Section 3.7 will be connected to an LLM API and wired up to retrieve live application, document, and allowance data, so that it can answer student status questions accurately.

- **System integration:** Authentication and role management, PHPMailer for notifications, Dompdf for letter generation, and the LLM API will all be integrated together so the three modules work as a single system rather than separate parts.

- **Testing and UAT:** Once the system is built, it will go through the three-stage testing process described in Section 4.5, unit testing, integration testing, and User Acceptance Testing, with particular attention on the Stage Gate Engine since it carries the main business rules.

- **Dissertation and technical paper:** The draft dissertation and technical paper will be prepared once development and testing are far enough along to report actual results, followed by the soft-bound submission.

- **VIVA preparation and defence:** The final project presentation and VIVA defence will be prepared based on the completed system, followed by any post-VIVA corrections before the hard-bound submission.

Completing these activities in FYP 2 would result in a fully functional prototype of the GRA/GA Student Lifecycle Management System, matching the objectives set out in Section 1.3 and the system design presented in Chapter 3 and Chapter 4 of this report.

REFERENCES

- Abdous, M., & He, W. (2008). A framework for process reengineering in higher education: A case study of distance learning exam scheduling and distribution. *International Review of Research in Open and Distance Learning, 9*(3), 1–12. https://doi.org/10.19173/irrodl.v9i3.535
- Al-Mashari, M., & Zairi, M. (1999). BPR implementation process: An analysis of key success and failure factors. *Business Process Management Journal, 5*(1), 87–112.
- Alabbas, M., Alomar, A., & Tayseer, K. (2024). A novel AI-powered Arabic chatbot framework for technical and vocational student helpdesk services and enhancing student interactions. *Applied Sciences, 14*(6), 2547. https://doi.org/10.3390/app14062547
- Balaji, S., & Murugaiyan, M. S. (2012). Waterfall vs. V-Model vs. Agile: A comparative study on SDLC. *International Journal of Information Technology and Business Management, 2*(1), 26–30.
- Davenport, T. H., & Short, J. E. (1990). The new industrial engineering: Information technology and business process redesign. *Sloan Management Review, 31*(4), 11–27.
- Hammer, M., & Champy, J. (1993). *Reengineering the corporation: A manifesto for business revolution*. Harper Business.
- Petrova, D., Nikolova, N., & Lengerov, A. (2024). Modeling business process reengineering in the digital era of Industry 4.0. *AIP Conference Proceedings, 2980*(1), 1–7. https://doi.org/10.1063/5.0185275
- Saad, A., Zaini, Z., & Alwahaibi, Y. (2024). Transforming medical assistance fund application management: A business process reengineering approach. *Journal of ICT in Education, 11*(1), 45–64. https://doi.org/10.37134/jictie.vol11.1.46.2024
- Sajja, R., Sermet, Y., Cikmaz, M., Cwiertny, D., & Demir, I. (2024). Artificial intelligence-enabled intelligent assistant for personalized and adaptive learning in higher education. *Information, 15*(10), 596. https://doi.org/10.3390/info15100596
- Torres Bermúdez, A. A. (2023). Collaborative framework for the implementation of digital transformation processes in higher education institutions: Case study Unicomfacauca. In *Decisioning Workshop 2023*.

APPENDICES

Appendix A: Proposal Defence Slide Extracts

This appendix is reserved for extracts from the FYP-1 Proposal Defence slide deck presented earlier in the semester, including the agenda, background of study, problem statement, objectives, scope of study, and system architecture slides that this report has been expanded from.

Appendix B: Link for clearer ERD diagram

https://kroki.io/mermaid/svg/eNrlWU1z2zYQvXvG_4GXzLSH_AHN9MDKtK2xYrmS3L
QnDkSAEhKSYABQjhr3v3fBTwAELcjTS6c-ZGJgASwWb9--
pQm_oWjPUX59dX314UPwy9RPPbteLaNzZtdXyir4cX0VwA8tZMBZRmKKg6eHZuyIe
HJAvBkvUE6ur_72Ov55E63PH6-s9OMrQbh-
vO7S7YNaoTuVVlnWOqUPkxzRzBwqkRAvjGNr9MAKuFWV7wi390ZJwqpCxkIiWYlm
DiNJJM1JkHAC_8Uxkr7h2Gyfb6LH7fmItIZ6UISsMAFXrLh0sRrHJUeS0yQumDkMQy
oOJePSnCiQpKxAGZWn4aLBkQoUk-8l5Sf7gJIzhUM78Bk5kixmaaw8Plk-
MUy0Ke0chHMqBDgQq1_NVQkqMIXhipO4JJwybLuyQ8VXBwbq4fYNuyU7BkhCBXh
PjpRVIoZQFIJxcaCluVqbcOytz-KK18GzzxDJgWXItfUw4dpamx229oPYOtpE4Xp-
H8wBP2uf3G8XxM0CHXIJ4I33RGC9STM3dj5FSZVJ691ZmtIEHk-
lmjNN_S53tw59sqc20y8CMB0yx3W_cfY0SySVGXFNuOkirQCoxT4WrOIJGYWMCRk3
ZzpS6QtJZAyoweNtm9E4YYVEiXR72jGUL0qW4XaxetzcL542nvXh9fXjR_ajpvRgFhyQ
6Klbzby-duwGk-
ylEC5stVs07zMDhirQngi_hw_Cp6flYl677QWBWFtggSFGZZnRpE6sMSw0pr19cIDo1rA
uEVfG1o5jQOkG8lQSg_2AYOH5oZZwoohZYaihQc0GiP6okghemsvpaVJgY7JPtO-
SFA3BWmyFSUJzlA2MiLKMvaBiQPBgwUomVMmzLVy37CBplU1R7XIq1WXtO9TTs
ANnx37WD8_hPLyJPi3mwW_P4XJx6w2Tfp2GD5RAtuVQJkfAcCBn_M7fKiihaWdSF0P
bhBYCiKXS38BRU-
1yA28AO__lfLpkXyLLU1w1HpwI4r5h_LxaPwTRH0_RehE9zj2Kh1oQDwtUGAcnFKg5J
YCSd4YyYXmJilNbZKxAMUH1SEjAd_CF7WJMRMJpaUSpzg932kylC2g7AX6ljCtSPk
JG-
gbx6flXf57SjM3gldVumqG8oqfvoBNON9_WNnPwC1Qu0ICjtK7kAbTOlIMNxswlmFFH
agC3jQ-lBSbfW85DOyS80_5mNX_-5CWoO0szyJglVU5c0sArwv1yV3hT2vdKjokSycOI-
qoyY8gNxyNkUuom1hr5HHQU_-pd_6E0rle_h0sPhmwtzcA1JI2yCwM3LJ1qWvpZ1e-
N4g1sJ-
yc72_urCPg4iV15EJd1CmeQdcYomPW1joxiNJJSyUdhUu5tCv6StVLrwlDm5LP2esMdM
62z6JZi9U3Pe6QM4M3Sgg9ElMxmibtwwuvLjq8i4K7cBv5vBDYxsrWBDCk0J7Ee4DGO3
O_2UDP77qcYJDqGS0saaZQoDe3xlnJVFNrpvnwScIubTXcMwTNRVWq37E_2j8tHm98
vtB0lmYQIfUUdXOXhNbDaxJACwaLAAZ5OF2R29NcbItJpvY8XdgHLaPt1uf-jZ15-
4yAiOXvhE-72G4E6gjsSUE4sgVyvXU_szu5Kj5O49RRXy-
Lyd2lrdao09IcvqzV8mvNbelX5xgsUg0QeTEl3tnGZdxaTXVV_0Kr4_0A_sIGon-
xtvkPSJtLoOqpZBqcXiZm_kdaZqKKawW0VwfaWGvV14dZzd-
jQq_N61pg4syWbWc93432c1vYcszWWPVbkje_TNlLWJqSpu-
5c7qqp58piqbMPWSRaXWBMgqXy9Xn0Ktz702tXOj48QKuPiv531IDOSsGulADei_Z0ffg
Fcr1b_n9t62M7umOqj9duJsj1cufrbN-
aTW_D7fB_WKzXa3_PB9mZR231makwXEHN4__nFPf4FtFhByzRf03iLGAUP9AGPLSu
x18vllsg-XqzgM3yjQGUws3FaZe1-m_PqCdVURabcg4hhVvq976-
izDMXBiRYzRAhSAMfr-
qNwuHiEBL24JbW4cMm0GgAaVAjLWyVe6oQdL6uYlOo1YxIDeTD3FuP_qHxO2IDxlP

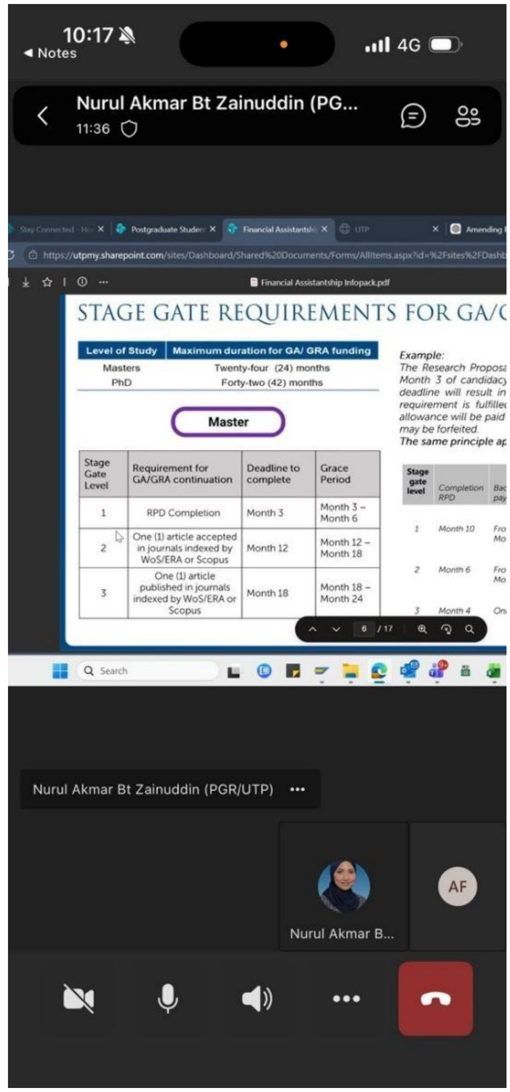
[BdtaDYkS4M1yeqUVn-JCn5SH-Gj7ivyz5OevTq62_rjMxb_APpchbM]

Appendix C: Supervisor, Uresearch teams & gra staff Consultation Records

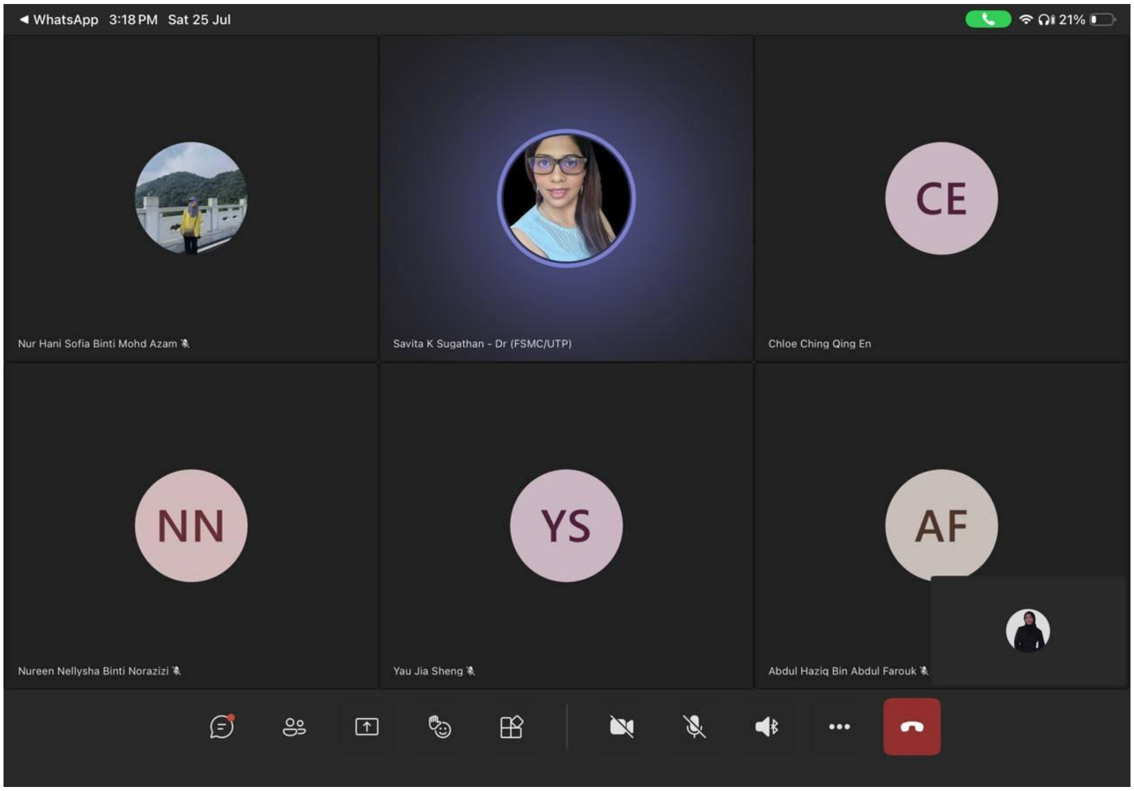
This appendix is reserved for photographic or written documentation of consultation sessions with the project supervisor, Uresearch 2.0 teams, and gra staff recording the feedback and guidance received throughout the proposal and interim reporting stages.

Meeting with supervisor and GRA staff

Meeting with assigned gra staff for respective modules



Meeting with supervisor with teams for updates



Meeting with ursearch team